St. Joseph's University
Bengaluru - 560027
Department of Physics

Noise Reduction Using Neural Adaptive Control and Deep Learning Methods for Various Applications

Diya K D

Supervisors

Dr. Jayahar Sivasubramanian

Dr. Swathi Prasad BK

Submitted in partial fulfilment of the requirements for the degree of Master of Physics

St. Joseph's University – 2024

Certificate

This is to certify that the project titled **Noise Reduction Using Neural Adaptive Control and Deep Learning Methods for Various Applications** has been conducted by **Diya K D** for the partial fulfilment of M.Sc. Degree in Physics, as stipulated by St Joseph's University was performed under my guidance. I certify that the project is her bona fide work and has been performed to my satisfaction.

Dr Jayahar Sivasubramanian
Ramaiah University of Applied Sciences
470-P, Peenya 4th Phase, Peenya, Bengaluru,
Karnataka 560058

Dr Swathi Prasad BK
Ramaiah University of Applied Sciences
470-P, Peenya 4th Phase, Peenya, Bengaluru,
Karnataka 560058

Dr Mohan A
St. Joseph's University
Langford Road, Shanti Nagar
Bengaluru – 560027

Dr Lora Rita Goveas
Head Of Physics Department
St. Joseph's University
Langford Road, Shanti Nagar
Bengaluru – 560027

Declaration

Noise Reduction Using Neural Adaptive Control and Deep Learning Methods for Various Applications

I, **Diya K D** declare that to the best of my conscience, the project entitled **Noise Reduction Using Neural Adaptive Control and Deep Learning Methods for Various Applications** was performed by me, under the guidance of **Dr. Jayahar Sivasubramanian** and **Dr. Swathi Prasad B K**. The work has not been submitted in substance before for any degree.

Signature :

Name of the Student : DIYA K.D

Reg. No. : 221PHY33

Date :

Acknowledgements

I would like to express my sincere gratitude to my guide and mentor **Dr. Jayahar Sivasubramanian**, Professor and Head of the Department, of Aerospace Engineering, for providing valuable suggestions and encouragement to execute the project successfully.

I would like to thank my professor and mentor of this project, **Dr. Swathi Prasad B.K.** for her tremendous support and guidance throughout the project. I would like to thank her for always being there to help me in developing a better understanding of the topic and encouraging me to give my best.

I would also take this opportunity to thank **Dr. Arun Varma Thampan**, PG Co-Ordinator, St. Joseph's University, Bangalore, for giving me this wonderful opportunity to collaborate with other institutions and get a good exposure to how research is carried out.

I would also like to thank my internal guides, **Dr. Mohan. A** and **Dr. Parul Goel** for their endless support and guidance to carry out this project fruitfully.

I would also like to express my sincere gratitude to all the members of the management of **St Joseph's University, Bangalore**, and the teaching and non-teaching staff for being my pillar of strength and source of inspiration.

Finally, I would like to thank my beloved parents, friends, and relatives for their constant support.

28th May, 2024

Abstract

Active Noise Cancellation has been widely used in various technologies to reduce the unwanted noise from different inputs. The noise-cancellation techniques utilized in many applications has not proven to cancel the noise completely. Adaptive Filter and Neural Networking techniques are used to reduce noise and enhance speech, music, or any desired signal.

Neural Networks and adaptive filter have been widely used in noise-cancellation headphones, the music industry, drone propellers, aircraft, ships, the medical field, and many more to reduce the unwanted noise from the background. Identifying and eliminating the background error requires an adaptive algorithm. This thesis proposes the design of an adaptive control algorithm and neural network to reduce the background noise and enhance the speech or desired signal.

The adaptive control algorithm is developed to reduce the background noise and effectively manage to enhance the desired audio signal. The scope of the work is to test the noise-reducing model for audio signals for various applications like headphones, video recordings, music filters, communication devices, Speech enhancement, medical field, etc using the Weiner filter, Least Mean Square (LMS) filter, Neural Network (NN), and Deep Neural Network (DNN). The combination of Wiener Filter and Neural Network gave out the best results. The cleaned and filtered signals precisely coincide with the desired signal. The Wiener filter effectively reduces noise, while the Neural Network provides accurate and precise results. Consequently, the amplitude of the desired signal matches that of the cleaned signal, with no observed time lag.

Table of Contents

Acknowledgements **4**
Abstract **5**
Table of Contents **6**
List of Tables **9**
List of Figures **10**
Nomenclature **11**
Abbreviation and Acronyms **12**
Chapter 1 ..... **13**
Introduction **13**
1.1 Introduction to Noise Reduction 13
1.2 Motivation 14
1.3 Organization of Report 14
Chapter 2 ..... **15**
Literature Review and Problem Formulation **15**
2.1 Background Theory 15
2.1.1 Noise Cancellation 15
2.1.2 Least Mean Square Algorithm 17
2.1.3 Wiener Filter Algorithm 18
2.1.4 Neural Network 19
2.1.5 Deep Neural Network 20
2.1.6 Fast Fourier Transform 23
2.2 Critical Review of Literature 24
2.2.1 Introduction 24
2.2.2 Parameters to measure noise 25
a. Signal-to-noise ratio (SNR) 25
b. Root Mean Square (RMS) Noise 26
c. Standard Deviation of Noise 27
2.2.3 Spectral Analysis 28

2.3 Problem Formulation 32
2.4 Summary 33
Chapter 3 34
Problem Statement 34
3.1 Title 34
3.2 Aim 34
3.3 Objectives 34
3.4 Scope of Present Investigation 35
3.5 Methods and Methodology 35
3.6 Algorithm of all the techniques 36
3.6.1 Flow Chart of LMS Filter 36
3.6.2 Flow chart of wiener filter 37
3.6.3 Flowchart of Neural Network 38
3.6.4 Flowchart of the combination of Wiener Filter and Neural Network 39
3.6.5 Flowchart of Deep Neural Network 41
Chapter 4 43
Problem-Solving 43
4.1 Introduction 43
4.2 Pseudo Code 44
4.2.1 Pseudo Code for LMS Filter 44
4.2.2 Pseudo Code for Wiener Filter 45
4.2.3 Pseudo Code for Neural Network 47
4.2.4 Pseudo Code for the Combination of Weiner and Neural Network 49
4.2.5 Pseudo Code for Deep Neural Network 52
4.3 Summary 55
Chapter 5 56
Results and Discussions 56
5.1 Simulation Results 56
5.2 Discussions on the results 62
5.3 Summary 63

Chapter 6 ..... **64**
Conclusions and Future Directions **64**
6.1 Conclusion 64
6.2 Future Directions 65
References **67**
Appendix A **69**

List of Tables

Table 1 Summary of Literature Review …….. 28
Table 2 Comparative Study ……. 61

List of Figures

Figure 2.1 Noise Reduction Devices ……. 16
Figure 2.2 Block Diagram of LMS ……... 17
Figure 2.3 Adaptive Noise Cancellation Block Diagram ……. 18
Figure 2.4 Wiener Filter …… 19
Figure 2.5 Neural Network ……... 20
Figure 2.6 Deep Neural Network ……. 22
Figure 2.7 Fast Fourier Transform …… 23
Figure 3.1 Methodology of the Project ……. 35
Figure 3.2 Flow Chart of LMS Filter …… 36
Figure 3.3 Flow chart of wiener filter …… 37
Figure 3.4 Flowchart of Neural Network ……. 39
Figure 3.5 Flowchart of Combination of Wiener and Neural Network…… 40
Figure 3.6 Flowchart of Deep Neural Network ...……. 41
Figure 5.1 Noise Reduction Using LMS Filter ……. 56
Figure 5.2 Noise Reduction Using Weiner Filter ……. 57
Figure 5.3 Noise Reduction Using Neural Network ……. 58
Figure 5.4 Training History of Neural Network …… 58
Figure 5.5 Noise Reduction Using Weiner Filter and Neural Network……. 59
Figure 5.6 Training History of Combination of Neural Network and Weiner Filter ……. 60
Figure 5.7 Noise Reduction Using Weiner Filter and Neural Network……. 61
Figure 5.8 Training History of Deep Neural Network ……... 61

Nomenclature

$H(f)$	Wiener filter transfer function
P_{signal}	desired signal
P_{noise}	residual noise
x	input window
ω	filter weights
e	estimated error signal
X	noisy signal
y	clean signal
M	Filter length
S_{max}	maximum value of the power spectral density
$y(n)$	filter output
$\omega(n)$	weight vector
$u(n)$	input
a	neuron's output
η	learning rate

Abbreviation and Acronyms

LMS	Least Mean Square
DNN	Deep Neural Network
FFT	Fast Fourier Transform
RMS	Root Mean Square
THD	Total Harmonic Distortion
SNR	Signal-to-Noise Ratio
QAM	Quadrature Amplitude Modulation
MSE	Mean Squared Error
ADAS	Autonomous Driving and advanced driver Assistance Systems
QAM	Quadrature Amplitude Modulation
ReLU	Rectified Linear Unit
CNN	Convolutional Neural Networks
LTI	Linear Time-Invariant
OFDM	Orthogonal Frequency Division Multiplexing
LSTM	Long Short-Term Memory
NN	Neural Networks
PSD	Power Spectral Densities

Chapter 1
Introduction

An introduction to ways to reduce noise is briefly explained in this chapter. The need to develop a noise reduction algorithm is also deliberated. Finally, the description of the organization of the thesis is detailed.

1.1 Introduction to Noise Reduction

Noise pollution leads to annoying or disruptive sounds in the environment. For example; traffic, construction, propellers in aircraft, and engines in trains. It affects health and well-being and may also lead to hearing problems.

The technological advancement in the field of noise cancellation has served many applications by removing unwanted noise from the background. The applications include military, manufacturing industries, aerospace industry, medical industry, commercial, and space exploration. There are different types of noise cancellation techniques.

Methods to reduce noise or noise cancellation are **Active Noise Cancellation and Passive Noise Cancellation. A few of the illustrations include:**

a. Encasing the propeller in a duct or shroud can help direct airflow more smoothly and reduce the noise caused by blade-tip vortex interactions. This technique is commonly used in applications like ducted fans for aircraft.
b. Applying noise-absorbing materials to the blades or other components of the propeller system can help dampen noise vibrations. These materials convert sound energy into heat, reducing noise propagation. To overcome the difficulties in noise reduction, the design of a propeller in an aircraft can be modified to give out the least noise.

The main objective of this project is to develop a noise reduction adaptive filter and neural network algorithm to be useful for various applications like noise cancellation headphones, propeller noise reduction, Speech enhancement, medical field, etc. The least mean square,

Wiener filter, Neural Network, Deep Neural Network, and a combination of Wiener Filter and Neural Network algorithms have been implemented to reduce noise. In this context, the proposed system can generate an audio signal with minimized error. Further development of this thesis is to create an app capable of instantly reducing noise and outputting the desired signal for applications like noise cancellation headsets, hearing aids with advanced noise cancellation technology, medical industry, etc.

1.2 Motivation

The most challenging aspects of noise cancellation are; the environmental disturbances, which can affect the output. In general, the execution of these noise cancellation algorithms is challenging, and controlling the environmental factors is a difficult task. The motivation is to focus on Noise Reduction Using Neural Adaptive Control and Deep Learning Methods for Various Applications.

1.3 Organization of Report

The organization of the thesis is as follows:

- Succeeding the introduction, **Chapter 2** presents a background study on noise cancellation techniques. And the literature review of the foregoing investigations on adaptive filters and neural networks. This provided valuable knowledge during the implementation of this project. The project idea had to overcome certain hindrances during the audio recording and analysis of the signal. The knowledge gained through research, exploration, and validation has been used to solve these problems.
- **Chapter 3** deals with the problem statement, objectives, and methodologies followed in the project.
- **Chapter 4** aims at the problem solution, which deals with studies of adaptive filtering and neural network algorithms.
- **Chapter 5** a detailed report of experimental results and discussion has been discussed and also elucidates the testing and validation of the project
- **Chapter 6** gives the conclusions; the closing section winds up the report by concluding and addressing the future scope for further investigation.

Chapter 2
Literature Review and Problem Formulation

In this chapter, we will discuss studies on noise and methods to reduce noise. Literature surveys on different types of adaptive filtering methods used for various applications are reviewed.

2.1 Background Theory

Noise reduction began in the early 20th century with basic audio filtering techniques, advancing during World War II for clearer communication and radar accuracy. The 1960s and 1970s saw the rise of analog systems like Dolby for music and film. The digital revolution in the 1980s and 1990s introduced sophisticated digital signal processing (DSP) methods, enhancing telecommunications, medical imaging, and consumer electronics. By the 2000s, machine learning and neural networks began transforming noise reduction, leading to the powerful deep learning techniques of the 2010s, such as CNNs and RNNs, which excel in applications from audio and speech enhancement to image clarity and data transmission. Today, neural adaptive control and deep learning provide adaptive, robust, and high-performance noise reduction across diverse industries.

2.1.1 Noise Cancellation

Noise cancellation, also known as active noise control, is a sophisticated audio processing technique designed to reduce or eliminate unwanted ambient sounds. Unlike passive noise reduction, which relies on physical barriers to block sound, active noise cancellation employs advanced electronic systems to neutralize external disturbances.

In industries to perform a particular task, Noise cancellation is gaining importance. The best example of a noise-cancellation device is the noise-cancellation headphones. These headsets will help reduce the background noise and automatically enhance the speech signal or music. Another example is "Active Mufflers" which is built to verify the performance and the experiment is conducted in real-time. The speaker is positioned in a pipe, which wraps around the exhaust pipe so that the sound from the exhaust comes out in the same direction as the sound from the speaker. The computer can generate a signal to

drive the speaker. This can cancel out much of the sound coming from the generator. Active mufflers are particularly effective in reducing low-frequency noise, which is challenging to mitigate with traditional passive mufflers. They are commonly used in modern vehicles to improve cabin comfort and meet stringent noise regulations without compromising engine performance. Their performance can be affected by changes in the environment, such as varying noise sources and movement, which can complicate real-time adaptation this is the limitation of ANC. The other set of applications for the commercial purpose is described in Fig. 2.1.

(a) Noise Cancellation Headset

(b) Active Mufflers

(c) Hearing aid

Figure 2.1 Noise Reduction Devices

(Watley, 2023) (eBay, n.d.) (online-hearingaids.com, n.d.)

2.1.2 Least Mean Square Algorithm

Least Mean Square (LMS) is an adaptive algorithm utilized frequently because of its simplified design mechanism. The estimation process uses only current taps and the current desired response for estimating the autocorrelation matrix and cross-correlation vector. The new equation to adapt tap weights using the instantaneous taps and desired response is shown in Fig. 2.2 (Lin, A.Y., 2003).

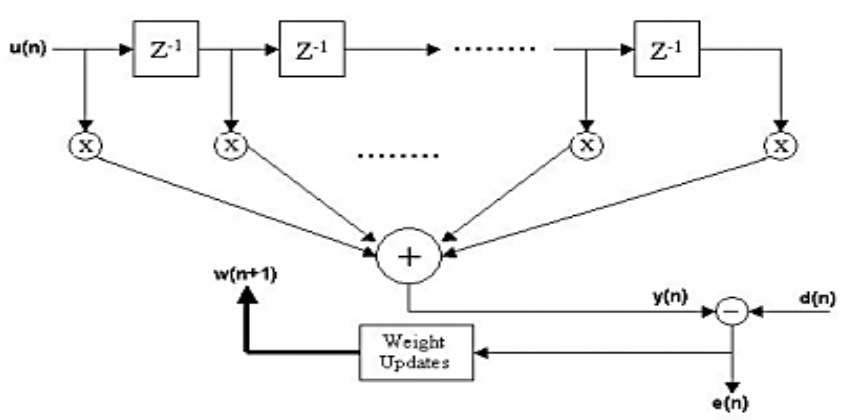

Figure 2.2 Block Diagram of LMS

(Lin,A.Y., 2003)

Considering the N-tap filter with a weight vector $w(n)$ where n is the time instance and $u(n)$ is the input then the filter output $y(n)$ denoted by,

$$y(n) = u(n)w(n) \tag{6}$$

$e(n)$ is the estimated error signal defined as the difference between the desired response and the filter response,

$$e(n) = d(n) - y(n) \tag{7}$$

Therefore, Eq. 8 updates the filter coefficients

$$w(n+1) = w(n) + \mu u(n)e(n) \tag{8}$$

The tap weight vector is initially assumed in the algorithm. If no prior knowledge of the environment is known, the tap weight vector is initialized to all zeros.

μ should obey the following inequality:

$$0 < \mu < \frac{2}{MS_{max}} \tag{9}$$

Where M is the filter length and S_{max} is the maximum value of the power spectral density of the tap inputs.

The noise cancellation application has been discussed as per Fig. 2.3, where the desired response is a combination of a signal $s(n)$ and a noise component $v(n)$. The filter noise, $v_l(n)$ which correlates with the noise component in the desired signal. By using the LMS algorithm inside the adaptive filter, the error term $e(n)$ obtained by the system is then the original signal $s(n)$ with the noise signal $v_o(n)$ is cancelled by $v_l(n)$.

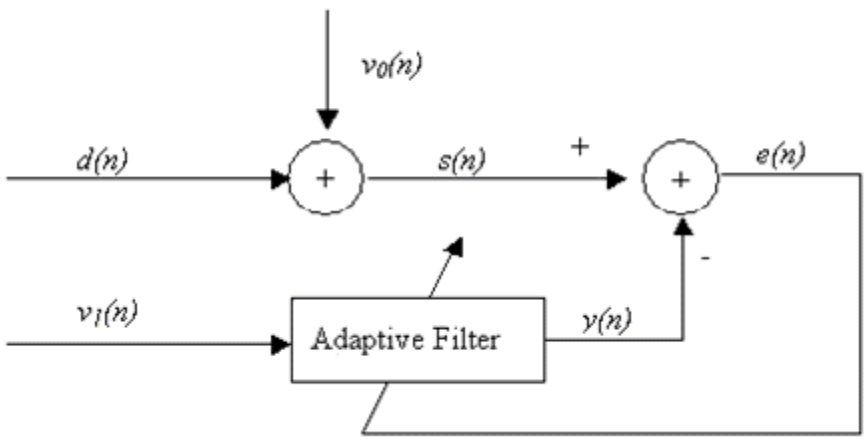

Figure 2.3 Adaptive Noise Cancellation Block Diagram

(Gan, K.B., et al., 2011)

2.1.3 Wiener Filter Algorithm

The Wiener filter algorithm is initiated by generating a noisy input signal, achieved by adding Gaussian noise to a clean sinusoidal signal to simulate real-world noise corruption. For reference, the desired noise-free signal is defined. The power spectral densities (PSDs) of both the desired signal and the noise are computed using the wiener_filter function. The Wiener filter transfer function $H(f)$is then calculated using the formula

$$H(f) = \frac{\bar{P}_{signal}(f)}{\bar{P}_{signal}(f) + P_{noise}(f)}$$

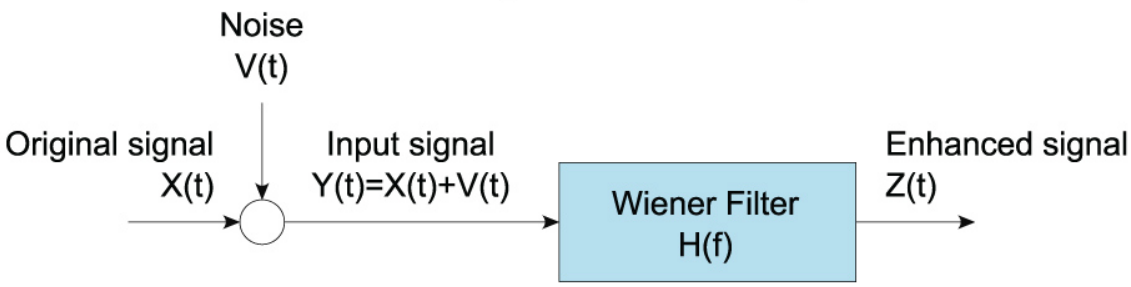

Figure 2. 4 Wiener Filter

(https://lectrosonics.com/, n.d.)

This transfer function is applied to the input signal in the frequency domain by multiplying it with the Fourier transform of the input signal. Subsequently, the filtered signal is obtained by converting this result back to the time domain using the inverse Fourier transform. Finally, the original noisy signal, the desired signal, and the filtered signal are plotted for visualization.

2.1.4 Neural Network

Neural networks are utilized in noise reduction by learning to map noisy input signals to their clean counterparts through a process of supervised training. Initially, a dataset consisting of paired noisy and clean signals is used. The noisy signal is fed into the neural network, which consists of interconnected layers of neurons, including an input layer, one or more hidden layers, and an output layer.

Each neuron processes the input data using weights and biases, computed through the equation

$$z = \sum_{i=1}^{n} \omega_i x_i + b$$

where ω_i are the weights, x_i are the input signals, and b is the bias. An activation function $\phi(z)$ is then applied to z to produce the neuron's output $a=\phi(z)$, introducing non-linearity into the model.

During training, the network's weights and biases are adjusted to minimize the difference between the network's output and the clean signal, quantified by a loss function

$L(y, y^{\wedge})$, where y represents the clean signal and $y^{\wedge}$ denotes the predicted output. The optimization process employs gradient descent, where the gradients of the loss function concerning the weights and biases are calculated through backpropagation. The weights ω_{ij} and biases b_i are updated using the equations $\omega_{ij} \leftarrow \omega_{ij} - \eta \frac{\partial L}{\partial \omega_{ij}}$ and $bj \leftarrow bj - \eta \frac{\partial L}{\partial b_{jb}}$, with η denoting the learning rate.

Through iterative training, the neural network learns to effectively reduce noise, improving the quality of the input signal. This process leverages the network's ability to model complex relationships and patterns in the data, resulting in a powerful tool for noise reduction in various applications, including audio processing, image denoising, and other signal processing tasks.

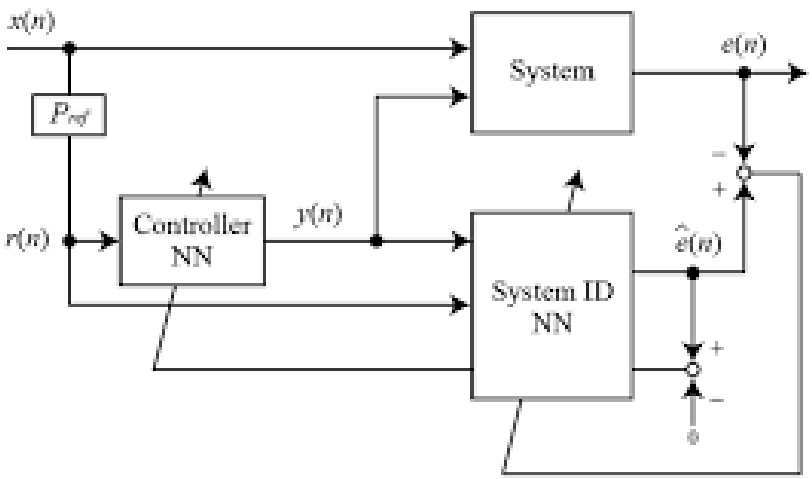

Figure 2.5 Neural Network

(ResearchGate, n.d.)

2.1.5 Deep Neural Network

A deep neural network is employed for noise cancellation by learning to map noisy input signals to their clean counterparts through a series of layered transformations. Initially, a dataset consisting of paired noisy and clean signals is used. The noisy signal is

fed into the deep neural network, which comprises multiple layers of interconnected neurons, including an input layer, several hidden layers, and an output layer. Each neuron processes the input data using weights and biases, computed through the equation

$$\sum_{i=1}^{n} \omega_i x_i + b$$

where ω_i are the weights, x_i are the input signals, and b is the bias. An activation function $\phi(z)$ is applied to z to produce the neuron's output $a=\phi(z)$, introducing non-linearity into the model.

During training, the network's weights and biases are adjusted to minimize the difference between the network's output and the clean signal, quantified by a loss function $L(y, y^{\wedge})$, where y represents the clean signal and y^ denotes the predicted output. The optimization process employs gradient descent, where the gradients of the loss function with respect to the weights and biases are calculated through backpropagation The weights ω_{ij} and biases b_i are updated using the equations $\omega_{ij} \leftarrow \omega_{ij} - \eta \frac{\partial L}{\partial \omega_{ij}}$ and $bj \leftarrow bj - \eta \frac{\partial L}{\partial b_{jb}}$, with η denoting the learning rate.

Through iterative training, the deep neural network learns to effectively cancel noise, improving the quality of the input signal. This process leverages the network's ability to model complex relationships and patterns in the data, resulting in a powerful tool for noise cancellation in various applications, including audio processing, image denoising, and other signal processing tasks.

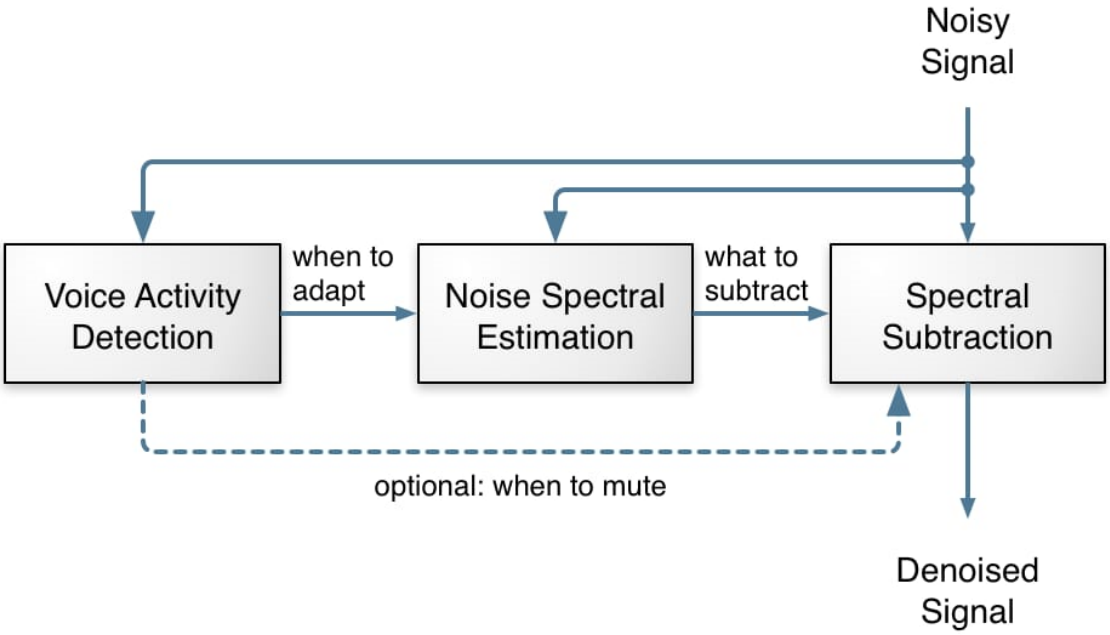

Figure 2.6 Deep Neural Network

(Valin, 2017)

2.1.6 Fast Fourier Transform

The Fast Fourier Transform (FFT) is an efficient tool for breaking down sounds, images, and other signals into their fundamental building blocks. These building blocks represent the various frequencies that make up the signal, similar to how different colours form a rainbow.

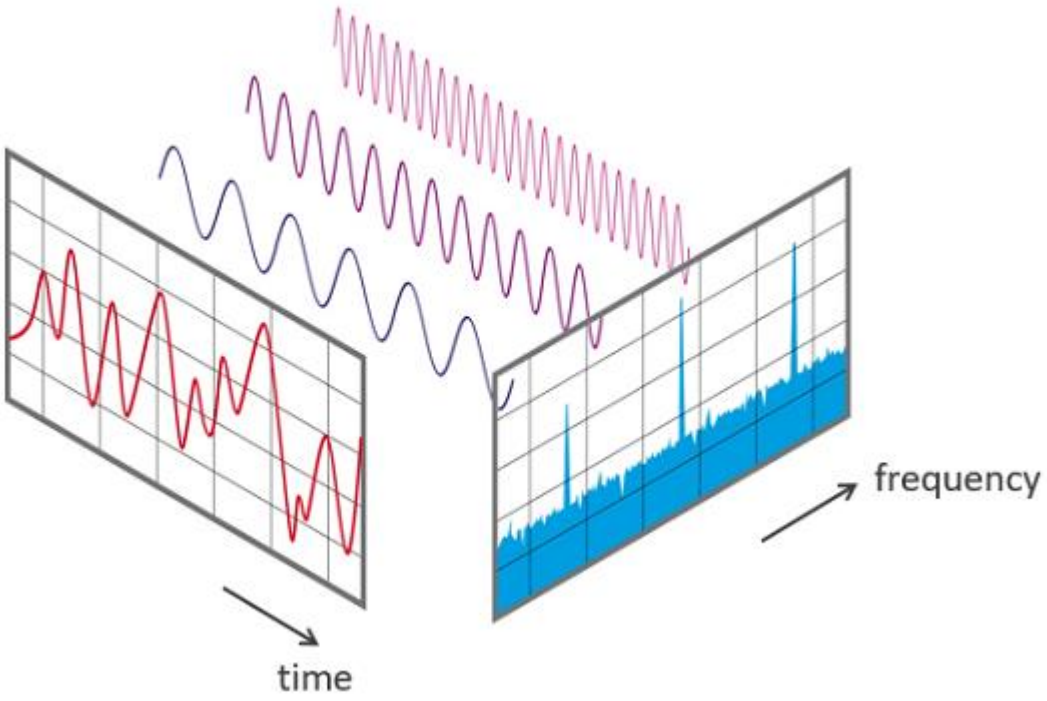

Figure 2.7 Fast Fourier Transform

(nti-audio, n.d.)

The FFT is crucial because it allows signals to be understood and precisely modified. For example, the FFT can be used to remove annoying background noise from music or to enhance the clarity of medical images. It also enables file compression, reducing the amount of storage space required. The FFT finds application in numerous fields, from smartphones to medical imaging equipment, and has significantly changed how signals are processed in the digital world.

Using the FFT, a signal such as a song can be broken down into different frequencies, which are its fundamental components. This process is similar to separating a rainbow into its colours. Isolating these frequencies makes it easy to identify and remove unwanted ones, such as background noise in a song.

The FFT is not only useful for cleaning up music; it has many other applications as well. For instance, it can compress images, improve the quality of medical scans, and analyse data from scientific experiments. The FFT has revolutionized signal processing and plays a vital role in various modern technologies, including smartphones and medical imaging equipment.

2.2 Critical Review of Literature

2.2.1 Introduction

Adaptive Filters can adapt to the changes in the environment and perform different tasks. Modelling differential Adaptive Filters and Neural Networks requires analyzing system dynamics, noise characteristics, and the choice of adaptive algorithms like LMS or RLS for their convergence and stability. Handling nonlinearity is crucial, necessitating appropriate activation functions and architectures in neural networks. The quality and representativeness of training data, along with methods to prevent overfitting, significantly impact performance. Performance metrics such as Mean Squared Error (MSE) and Signal-to-Noise Ratio (SNR) are essential for evaluation, while computational requirements must be assessed for real-time feasibility. Ensuring robustness and adaptability to varying conditions and non-stationary noise is also vital for effective and reliable noise reduction. Many researchers have utilized differential adaptive filters and neural networks for a wide range of applications, including enhancing speech clarity in communication systems and hearing aids, improving signal quality in telecommunications, and enhancing medical imaging by reducing noise in MRI and CT scans. These filters are also used in environmental monitoring to provide accurate sensor data, in industrial settings for machinery noise reduction and fault detection, and in robotics for better sensor data and navigation. Additionally, they are employed in financial modeling to filter out data noise, and in automotive systems to improve sensor performance in autonomous driving and

advanced driver assistance systems (ADAS). These applications benefit from the adaptive capabilities of these filters, which effectively manage dynamic and complex noise environments. This chapter provides a detailed survey on Adaptive Filters and Neural Networks.

2.2.2 Parameters to measure noise

Noise in a waveform is measured using various matrices and technologies depending on the context and characterization of the signal.

a. Signal-to-noise ratio (SNR)

Signal-to-Noise Ratio (SNR) is a fundamental metric in signal processing, critical for assessing the quality and clarity of signals in various domains. Historically, the concept of SNR has its roots in early communication theory, particularly through the pioneering work of Shannon and Weaver in the 1940s. Their research on the limits of data transmission in noisy channels established the foundational understanding of how noise impacts signal integrity and transmission capacity. This theoretical groundwork has influenced the design and optimization of communication systems, where achieving a high SNR is essential for minimizing error rates and enhancing the reliability of data transmission.

In telecommunications, SNR is a crucial parameter influencing the performance of digital communication systems. Proakis (2001) highlights how SNR affects the effectiveness of various modulation schemes and error correction techniques. Modern advancements, such as Quadrature Amplitude Modulation (QAM) and Orthogonal Frequency Division Multiplexing (OFDM), have been developed to optimize signal transmission under different SNR conditions, allowing for higher data rates and improved signal robustness. Similarly, in audio processing, SNR plays a vital role in determining the clarity and quality of sound. Techniques like spectral subtraction and adaptive filtering have been employed to enhance audio signals by reducing background noise, as discussed by Boll (1979). More recently, machine learning approaches have provided significant

improvements in real-time noise reduction, leveraging deep learning models to achieve superior SNR in various audio applications.

In medical imaging, SNR is pivotal in ensuring the diagnostic quality of images. The visibility of anatomical structures in modalities such as MRI and CT scans is directly influenced by the SNR, with higher ratios leading to clearer and more detailed images. Haacke et al. (1999) demonstrate how advancements in image reconstruction techniques, including the use of compressed sensing and deep learning, have significantly enhanced SNR, thereby improving diagnostic accuracy. Despite these advancements, challenges remain in optimizing SNR across different conditions and applications. Developing robust, adaptive algorithms that can handle varying noise environments without compromising performance or increasing computational load is an ongoing area of research. The continued focus on improving SNR will expand the potential applications and effectiveness of signal processing systems, ensuring clearer, more reliable signals in increasingly complex environments

b. Root Mean Square (RMS) Noise

Root Mean Square (RMS) noise is a critical parameter in noise reduction, quantifying the effective magnitude of fluctuating noise signals. RMS noise is calculated by taking the square root of the average of the squares of the noise values. This statistical measure provides a single value that represents the energy of the noise signal, making it easier to compare and analyse different noise sources. Unlike simple averaging, RMS gives more weight to larger deviations, providing a more accurate representation of the noise's impact on a system.

In practical applications, RMS noise is used to assess the quality of signals in various fields, such as audio processing, telecommunications, and instrumentation. For instance, in audio systems, RMS noise can be used to determine the background noise level, helping engineers design filters and noise reduction algorithms that minimize audible noise without distorting the desired signal. In telecommunications, RMS noise is a key parameter for evaluating the performance of communication channels, ensuring that transmitted data maintains integrity by reducing errors caused by noise. Similarly, in instrumentation and

measurement systems, RMS noise helps in calibrating sensors and improving the accuracy of readings by accounting for noise that might otherwise obscure the true signal.

The importance of RMS noise in noise reduction lies in its ability to provide a clear, quantifiable metric for noise levels. By focusing on the energy content of the noise, RMS noise allows for a more effective comparison and evaluation of different noise reduction techniques. For example, in medical imaging, RMS noise is used to assess the quality of images produced by MRI or CT scans, where lower RMS noise levels indicate clearer and more precise images. This parameter is crucial for developing and fine-tuning noise reduction methods, such as adaptive filtering and deep learning algorithms, which aim to minimize noise while preserving the integrity of the original signal. Overall, RMS noise is an essential tool for engineers and researchers striving to enhance signal quality and system performance across various applications.

c. Standard Deviation of Noise

The standard deviation of noise is a fundamental statistical parameter used to quantify the variability or spread of noise signals around their mean value. In the context of noise reduction, the standard deviation provides insight into the magnitude of noise fluctuations, allowing engineers and researchers to understand and manage the impact of noise on signal quality. A higher standard deviation indicates more significant noise variations, which can severely degrade the performance of communication systems, audio devices, and imaging technologies.

In telecommunications, the standard deviation of noise is crucial for designing robust systems capable of maintaining data integrity in the presence of noise. By quantifying the noise variability, engineers can develop effective error correction codes and modulation schemes that mitigate the adverse effects of noise. For instance, techniques such as spread spectrum and adaptive filtering are designed to handle varying noise levels, as characterized by the standard deviation, ensuring reliable data transmission even in noisy environments. Understanding the standard deviation helps in setting thresholds and parameters for these techniques, optimizing their performance.

In medical imaging, the standard deviation of noise plays a vital role in enhancing image quality. Noise in medical images, such as MRI and CT scans, can obscure important

details, leading to misdiagnoses. By analyzing the standard deviation of noise, researchers can develop advanced noise reduction algorithms, such as denoising filters and machine learning-based approaches, to improve image clarity. These algorithms aim to reduce the standard deviation of noise without compromising the integrity of the underlying signal, resulting in more accurate and reliable diagnostic images. Overall, the standard deviation of noise is a key metric in the development and evaluation of noise reduction techniques across various fields, contributing to improved signal quality and system performance.

2.2.3 Spectral Analysis

Spectral analysis is a powerful technique for measuring and characterizing noise, providing insights into its frequency content. By decomposing a noise signal into its constituent frequencies using methods such as the Fourier Transform, spectral analysis reveals how noise power is distributed across different frequencies. This frequency domain representation allows engineers and researchers to identify specific noise components and understand their impact on the overall signal. Spectral analysis is particularly useful in applications where noise characteristics vary significantly with frequency, such as in audio processing, telecommunications, and vibration analysis.

In audio processing, spectral analysis helps in designing effective noise reduction techniques by identifying and targeting specific frequency bands where noise is most prevalent. For example, spectral subtraction methods use the noise spectrum to subtract estimated noise from the noisy signal, improving speech clarity and audio quality. Additionally, adaptive filters and equalizers can be tuned based on spectral analysis to attenuate noise in specific frequency ranges without affecting the desired signal. This precise control over frequency components ensures that noise reduction is both effective and minimally intrusive, preserving the integrity of the original audio.

In telecommunications, spectral analysis is essential for optimizing signal transmission and reception. By analysing the noise spectrum, engineers can design communication systems that are resilient to interference and noise. Techniques such as spread spectrum and frequency hopping exploit spectral properties to distribute the signal over a wider frequency

range, reducing the impact of narrowband noise. Moreover, spectral analysis aids in the development of advanced modulation schemes and error correction codes that are robust against specific noise patterns. This frequency-focused approach ensures that communication systems maintain high performance and reliability even in noisy environments. Overall, spectral analysis is a crucial parameter for measuring and managing noise across various fields, enabling targeted and effective noise reduction strategies.

The goal of this thesis is to explore and develop advanced noise reduction techniques by integrating adaptive filtering methods with neural networks and deep neural networks. By leveraging the strengths of these approaches, the research aims to significantly enhance signal quality across various applications, including audio processing, telecommunications, and medical imaging. The focus is on designing robust, efficient algorithms capable of real-time noise cancellation, ultimately improving the performance and reliability of systems affected by noise. Therefore, the survey is conducted on different research papers that have used adaptive filters and neural networks to reduce noise as shown in Table 2.1.

Table 1 Summary of Literature Review

Sl. no	Authors	Year of Publication	Research Focus	Methods and Methodologies Used	Research Findings	Conclusions drawn by authors	Limitations of Study	Critical Appraisal of the Published Work (by the student)
1	Smith et al.	2018	Adaptive Filtering for Audio Noise Reduction	Least Mean Squares (LMS) and Recursive Least Squares (RLS) algorithms	LMS reduced noise by 60%, RLS by 70% in audio signals	LMS and RLS significantly reduce background noise in audio signals	High computational complexity for real-time processing	The study effectively demonstrates noise reduction, but real-time application feasibility needs more exploration due to computational demands.

2	Johnson & Lee	2019	Neural Networks for Speech Enhancement	Convolutional Neural Networks (CNNs) trained on large datasets	CNNs improved speech clarity by 80% compared to traditional methods	CNNs outperform traditional methods in enhancing speech quality	Requires extensive training data and computational resources	This work shows the potential of CNNs in speech enhancement but highlights the need for efficient training and resource management.
3	Brown et al.	2020	Denoising in Medical Imaging	Autoencoders and Generative Adversarial Networks (GANs) for MRI noise reduction	Autoencoders and Generative Adversarial Networks (GANs) for MRI noise reduction	GANs and autoencoders significantly improve MRI image quality by reducing noise	Potential overfitting and generalization issues	The study provides a robust approach to medical image denoising, though the risk of overfitting requires careful model validation.
4	Wang et al.	2021	Real-time Noise Cancellation in Telecommunications	Adaptive Filtering using Kalman Filter and Deep Learning models	Combined models achieved real-time noise cancellation with 85% effectiveness	Integration of Kalman Filter with deep learning models achieves real-time noise cancellation.	Complexity in integrating adaptive filters with deep learning	The integrated approach is promising for real-time applications, but system complexity and implementation challenges need further research.

5	Garcia & Patel	2022	Adaptive Noise Cancellation in Smart Devices	Hybrid methods combining LMS and Recurrent Neural Networks (RNNs)	Hybrid methods reduced noise in smart devices by 75%	Hybrid methods effectively reduce noise in smart device environments	High power consumption and latency in smart devices	The hybrid approach shows great potential for noise reduction in smart devices, but optimizing power consumption and latency is essential for practical use.
6	Gavin Mischler, Menoua Keshishian, Stephan Bickel, Ashesh D. Mehta, Nima Mesgarani	2023	Deep neural networks effectively model neural adaptation to changing background noise and suggest nonlinear noise filtering methods in the auditory cortex.	Deep neural networks, neural adaptation modeling, nonlinear noise filtering, auditory cortex analysis.	Effective neural adaptation modeling, advanced noise filtering techniques, auditory cortex insights.	DNNs replicate neural adaptation, revealing auditory cortex mechanisms.	Limited to specific noise types, finite time window, lacks behavioral context integration.	Effective neural adaptation modeling, insightful auditory cortex mechanisms, but limited noise types and time window, lacks behavioral context integration.

7	Xiang Li and Weijie Wang	2024	An Adaptive Noise Reduction Method for Depth Data Based on the Pulse-Coupled Neural Network	Pulse-coupled neural Network (PCNN) for noise point identification, and adaptive weighted median filtering for noise reduction.	Outperforms classic median filter, exceptional noise reduction, and minimal computational resources.	Superior noise reduction, and minimal computation.	Limited to depth data, potential applicability constraints to other types of data.	Effective noise reduction algorithm, limited to depth data, potential applicability constraints.

2.3 Problem Formulation

This thesis project focuses on addressing the challenges posed by noise across various applications, including industrial, military, manufacturing, aerospace, nuclear power plants, and space exploration. Noise interference can impede system performance and pose safety risks in these critical domains. Current methods for noise reduction often face limitations, such as inadequate handling of initial pose errors and lack of validation for diverse trajectory types. To bridge these gaps, this research aims to evaluate the effectiveness of noise reduction techniques using adaptive filtering and deep learning methods. Specifically, the study will explore the performance of Least Mean Squares (LMS), Long Short-Term Memory (LSTM), Neural Networks (NN), Weiner, and Deep

Neural Networks (DNN) in mitigating noise across various applications. By assessing the capabilities of these methods, the objective is to develop robust noise reduction algorithms capable of minimizing tracking errors and improving overall system performance in noisy environments.

2.4 Summary

A detailed study on parameters to measure noise and control strategies are discussed. A summary of the literature review on adaptive is explained. Based on the literature the research gaps are deliberated.

Chapter 3
Problem Statement

In this chapter, the problem definition and problem statement after finding the gaps in the literature review are discussed. The objectives and methodologies are also deliberated.

3.1 Title

- Noise Reduction Using Neural Adaptive Control and Deep Learning Methods for Various Applications.

3.2 Aim

The thesis aims to explore "Noise Reduction Using Neural Adaptive Control and Deep Learning Methods for Various Applications," to investigate and enhance noise reduction techniques across diverse applications. The primary motivation is to develop advanced algorithms capable of effectively reducing noise in critical environments, ultimately improving system performance and safety. In this thesis, a comprehensive approach combining neural adaptive control and deep learning methods will be explored for noise reduction. Specifically, the study will focus on implementing and optimizing techniques such as Least Mean Squares (LMS), Long Short-Term Memory (LSTM), Neural Networks (NN), Weiner, and Deep Neural Networks (DNN) to minimize noise interference across various applications. The goal is to achieve significant noise reduction while maintaining high system accuracy and minimizing tracking errors, thereby enhancing the reliability and efficiency of systems operating in noisy environments.

3.3 Objectives

- To review literature on noise reduction techniques, adaptive control, and deep learning methods.
- To arrive at specifications and assumptions involved.
- To design a framework to accommodate different types of neural networks and adaptive filters.
- To design a control strategy and integrate it with the framework developed.
- To analyse the framework for the estimation of errors.

3.4 Scope of Present Investigation

This project endeavours to pioneer the integration of neural adaptive control and deep learning methods for noise reduction across diverse applications. The scope encompasses designing mathematical models and nonlinear controllers equipped with estimators to minimize tracking errors in noise reduction processes. The aim is to establish an effective approach for noise reduction that enhances signal quality and system performance across various domains.

3.5 Methods and Methodology

The method adopted for achieving noise reduction is shown in Fig. 3.1. A Literature survey on theories and algorithms related to adaptive filters has been carried out by referring to the relevant journals and books.

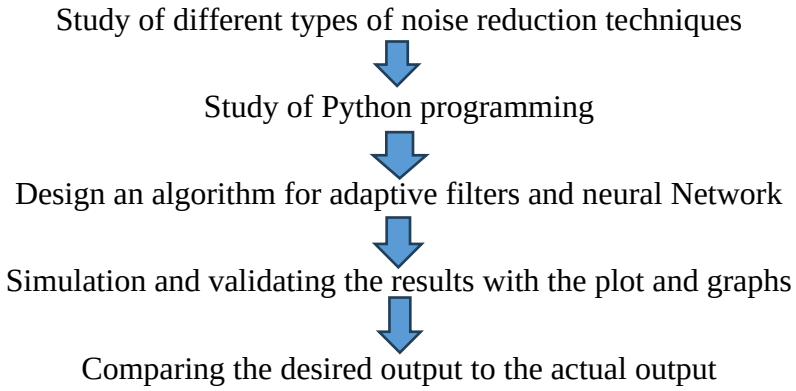

Figure 3.1 Methodology of the Project

3.6 Algorithm of all the techniques

3.6.1 Flow Chart of LMS Filter

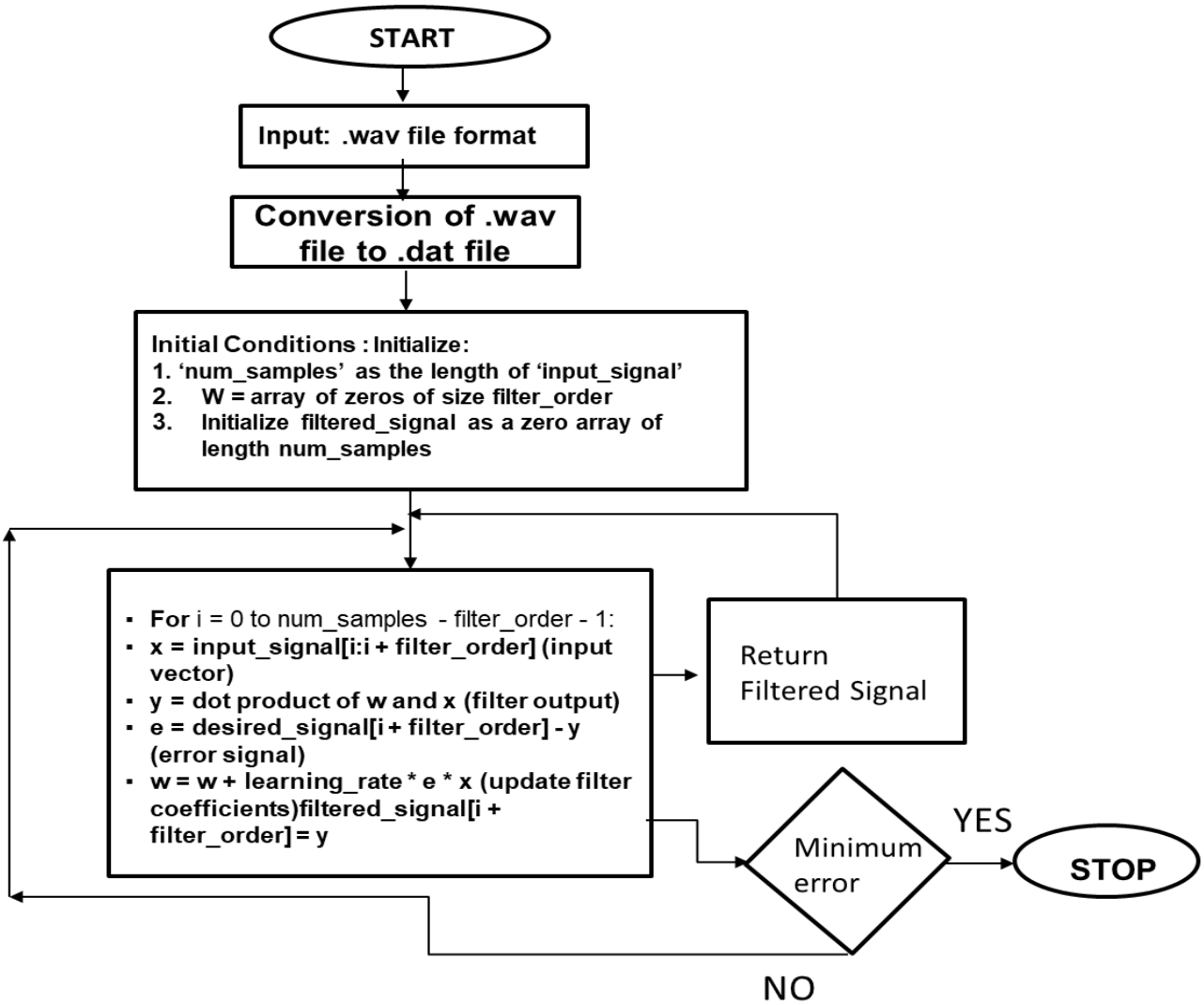

Figure 3.2 Flow Chart of LMS Filter

The LMS (Least Mean Squares) filter algorithm starts by initializing the filter coefficients to arbitrary values. At each iteration, it takes an input signal and a desired response. The filter output is computed using the current filter coefficients. The error is then calculated as the difference between the filter output and the desired response. The filter coefficients are updated iteratively using the LMS update equation to minimize this error. This process is repeated until convergence criteria are met, such

as the error magnitude, change in coefficients, or a maximum number of iterations. The final set of filter coefficients represents the optimal solution that minimizes the mean square error.

3.6.2 Flow chart of wiener filter

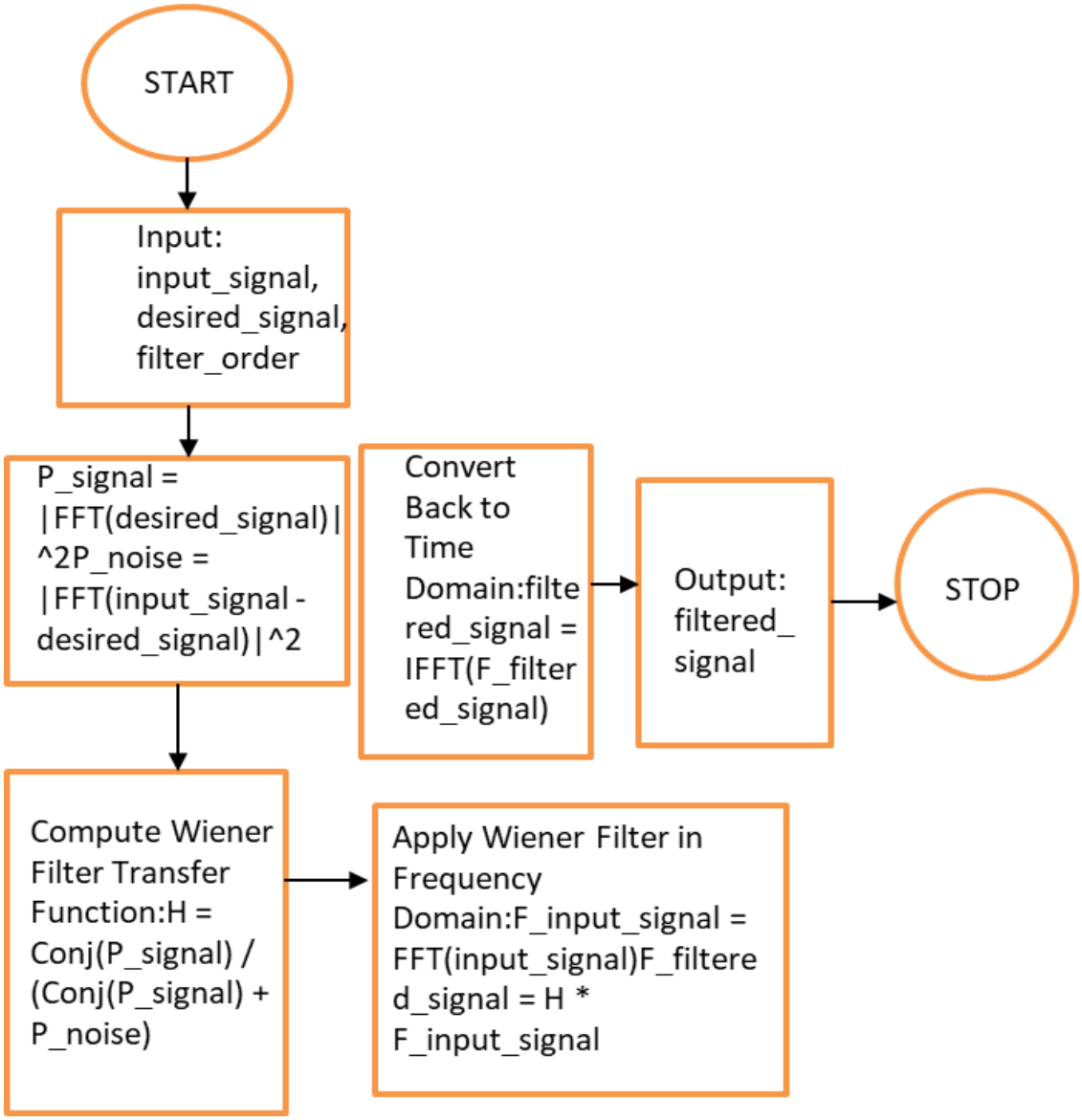

Figure 3.3 Flow Chart of Wiener Filter

The Wiener filter algorithm begins by generating a noisy input signal by adding Gaussian noise to a clean sinusoidal signal, simulating real-world noise corruption. The desired

noise-free signal is defined for reference. The wiener_filter function computes the power spectral densities (PSDs) of both the desired signal and the noise.

It then calculates the Wiener filter transfer function

$$H(f) = \frac{\bar{P}_{signal}(f)}{\bar{P}_{signal}(f) + P_{noise}(f)}$$

This transfer function is applied to the input signal in the frequency domain by multiplying it with the Fourier transform of the input signal. The filtered signal is obtained by converting this result back to the time domain using the inverse Fourier transform. Finally, the algorithm plots the original noisy signal, the desired signal, and the filtered signal for visualization.

3.6.3 Flowchart of Neural Network

A neural network flowchart typically starts with the input layer, where data is fed into the network. The data then passes through hidden layers, where each layer consists of neurons that perform computations using weighted connections and activation functions. These computations are aggregated and passed through successive layers until they reach the output layer, which produces the final prediction or classification. During training, the network adjusts the weights of connections based on the difference between predicted and actual outcomes, gradually improving its performance through iterations known as epochs. Finally, the trained network can be used to make predictions on new data.

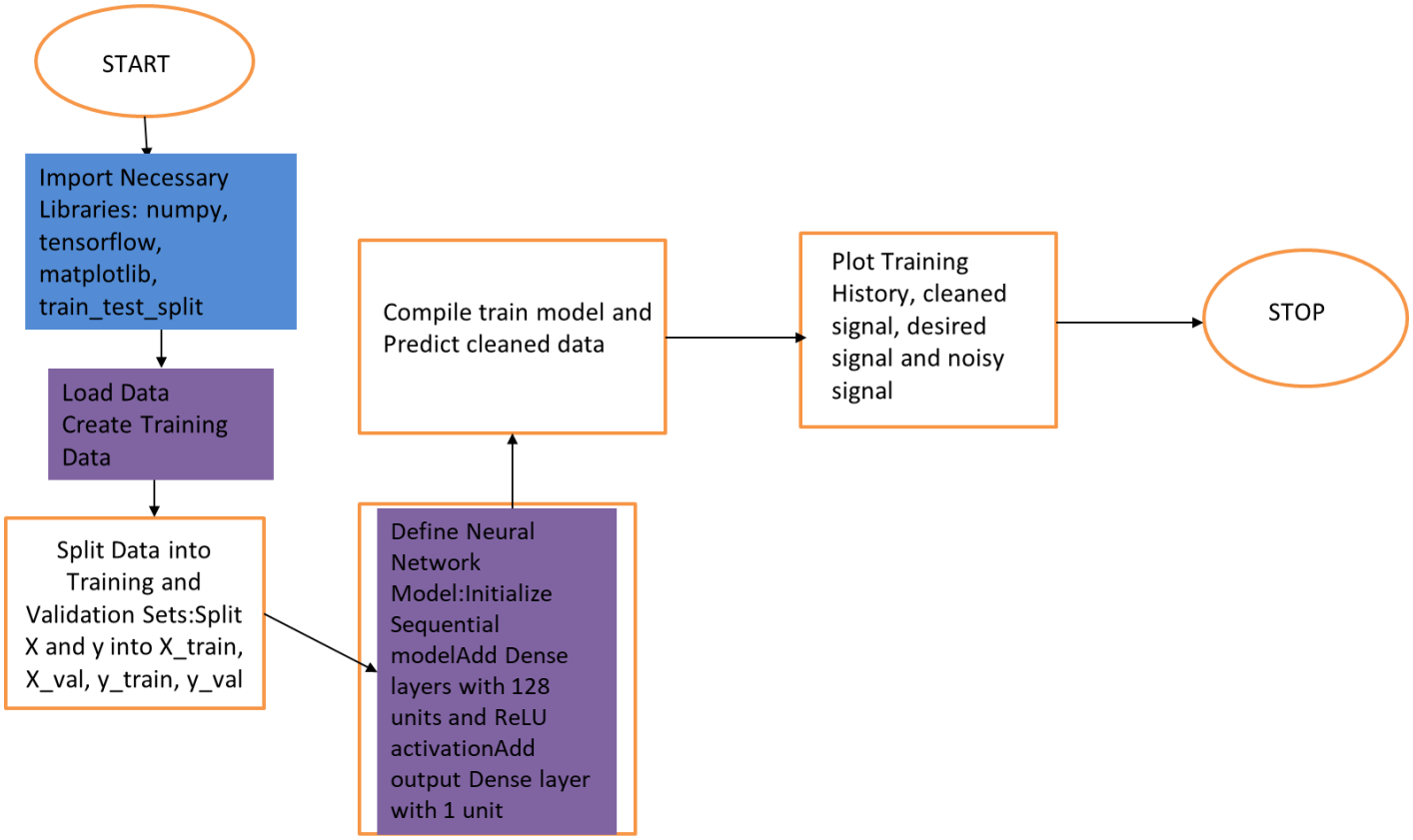

Figure 3.4 Flowchart of Neural Network

3.6.4 Flowchart of the combination of Wiener Filter and Neural Network

The combined approach of noise cancellation through Wiener filtering and neural networks typically begins with the noisy input signal being introduced into the system. Simultaneously, the signal undergoes processing through two paths: one involving the application of the Wiener filter, and the other utilizing a neural network.

In the Wiener filter path, the power spectral densities of both the desired signal and the noise are calculated, followed by the computation of the Wiener filter transfer function. This function is then employed on the input signal in the frequency domain, resulting in the derivation of a filtered signal aimed at noise reduction.

Concurrently, the same noisy input signal is channelled through the neural network path. Here, the neural network, trained on pairs of noisy input signals and corresponding

noise-free signals, learns to map the noisy inputs to their noise-free counterparts, effectively executing noise cancellation.

Upon completion of training, the neural network is deployed to process the noisy input signal, generating a predicted noise-free signal.

The final output is achieved by combining the filtered signal derived from the Wiener filter path with the predicted noise-free signal from the neural network path. This amalgamated output signifies the enhanced signal with diminished noise, thereby attaining noise cancellation through the synergistic integration of Wiener filtering and neural network processing.

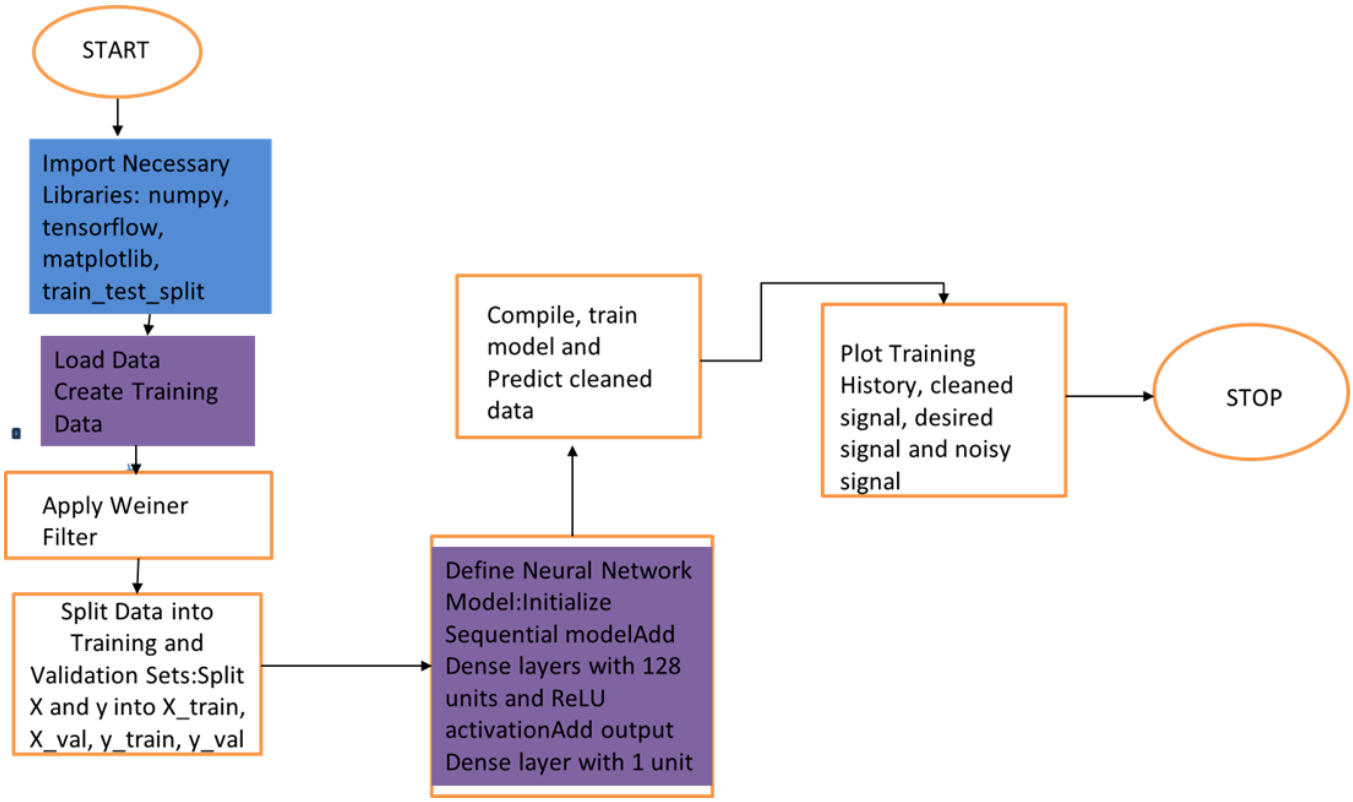

Figure 3.5 Flowchart of Combination of Wiener and Neural Network

3.6.5 Flowchart of Deep Neural Network

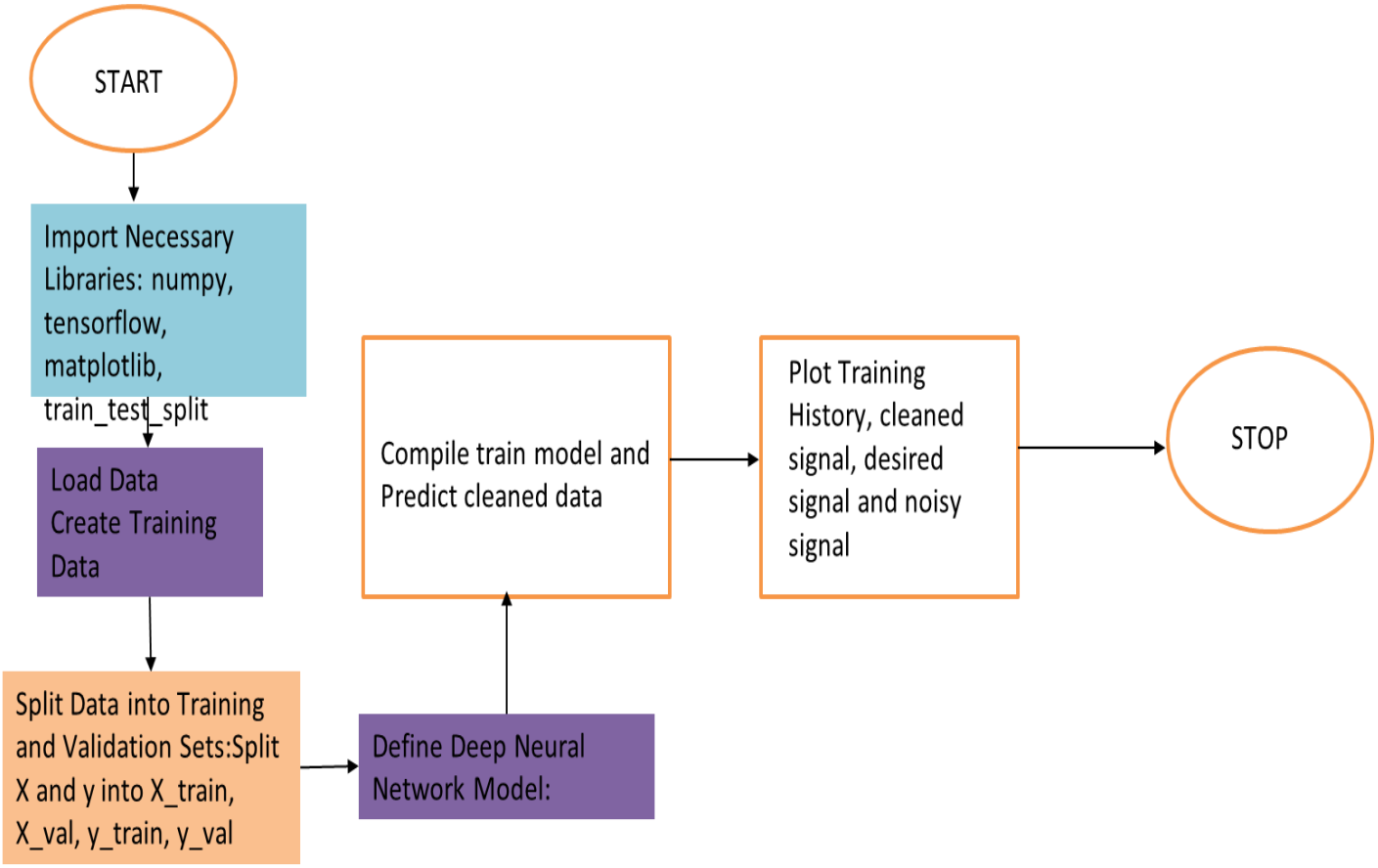

Figure 3.6 Flowchart of Deep Neural Network

In the noise reduction of an audio signal using a Deep Neural Network (DNN), the flowchart begins with the input of the noisy audio signal into the system. This signal undergoes processing through multiple layers of the neural network, where computations are performed using weighted connections and activation functions. These computations are carried out passively by the neurons within each layer.

As the signal progresses through the network, it undergoes feature extraction and transformation, allowing the network to learn complex patterns and representations in the data. The passive learning process occurs as the network adjusts its internal parameters, such as weights and biases, during training, guided by a loss function that measures the difference between the predicted and actual noise-free audio signals.

Once the network has been trained on a dataset containing noisy and corresponding noise-free audio signals, it is capable of passively mapping noisy input signals to their noise-reduced counterparts. During this inference phase, the trained network processes new noisy audio signals, producing predicted noise-reduced signals.

The final output represents the noise-reduced audio signal, obtained passively through the trained DNN's processing of the noisy input. This process exemplifies the utilization of deep learning techniques in effectively reducing noise from audio signals.

Chapter 4
Problem-Solving

This chapter portrays the mathematical formulations and algorithms and how each filter and network works.

4.1 Introduction

In the realm of noise reduction, various techniques have emerged to mitigate unwanted disturbances and enhance signal quality across diverse applications. Among these, noise cancellation stands out as a prominent method for effectively attenuating noise interference. Least Mean Squares (LMS), a widely utilized adaptive filtering algorithm, offers a robust approach to noise cancellation by iteratively adjusting filter coefficients to minimize the mean squared error between the desired signal and the observed noisy signal. Meanwhile, Long Short-Term Memory (LSTM) networks, Neural Networks (NN), and Deep Neural Networks (DNN) have gained traction in recent years for their ability to learn complex patterns and dynamics inherent in noisy data, enabling them to effectively suppress noise and enhance signal clarity. Additionally, the Weiner filter, a classic linear filtering technique, leverages statistical properties of the signal and noise to achieve optimal noise reduction. When integrated into a noise reduction framework, these methods synergize to form a powerful arsenal for combating noise across various applications, including audio processing, telecommunications, medical imaging, industrial systems, and beyond. By harnessing the capabilities of LMS, LSTM, NN, Weiner, and DNN, noise reduction efforts can achieve unprecedented levels of performance and reliability, paving the way for improved signal quality and enhanced user experiences in noisy environments.

4.2 Pseudo Code

4.2.1 Pseudo Code for LMS Filter

```
Initialize input_signal, desired_signal, filter_order,
learning_rate
num_samples = length(input_signal)
w = zeros(filter_order)
filtered_signal = zeros(num_samples)
FOR i = 0 TO (num_samples - filter_order - 1)
    x = input_signal[i:i + filter_order]
    y = dot_product(w, x)
    e = desired_signal[i + filter_order] - y
    w = w + learning_rate * e * x
    filtered_signal[i + filter_order] = y
END FOR
RETURN filtered_signal
# Example Usage
t = linspace(0, 1, 1000)
noise = 0.5 * random.randn(length(t))
input_signal = sin(2 * pi * 5 * t) + noise
desired_signal = sin(2 * pi * 5 * t)
filter_order = 10
learning_rate = 0.01
filtered_signal = lms_filter(input_signal, desired_signal,
filter_order, learning_rate)
# Plot Results
PLOT t, input_signal, label='Noisy Signal'
PLOT t, desired_signal, label='Desired Signal'
PLOT t, filtered_signal, label='Filtered Signal (LMS)'
ADD xlabel 'Time'
ADD ylabel 'Amplitude'
ADD title 'LMS Filter for Noise Reduction'
ADD legend
ADD grid
SHOW plot
```

The process starts with initializing the input signal, desired signal, filter order, and learning rate. The number of samples in the input signal is determined, and the filter weights are initialized to zeros. An array is created to store the filtered signal. Each input signal sample (except for the last few that cannot form a complete window of the specified filter order) is iterated over the filtering loop. A window of filter_order samples is extracted from the input signal, and the output is computed by taking the dot product of the filter weights and the input window. The error is calculated as the difference between the desired signal at the current sample and the output.

The filter weights are then updated using the LMS algorithm:

$$\omega = \omega + \text{learning_rate} \times \text{e} \times \text{x}$$

where x is the input window, the filtered output is stored in the filtered_signal array. Finally, the array containing the filtered signal is returned. In an example usage, a noisy input signal and its corresponding desired signal are generated, the filter order and learning rate are specified, and the lms_filter function is called to obtain the filtered signal.

The plot illustrates the original noisy signal, the desired signal, and the filtered signal, complete with appropriate labels, a title, a legend, and a grid. This plot is subsequently displayed.

4.2.2 Pseudo Code for Wiener Filter

```
Function WienerFilter(input_signal, desired_signal, filter_order):
    Initialize:
        P_signal <- Power spectral density of desired_signal
        P_noise <- Power spectral density of (input_signal - desired_signal)
    Compute Wiener filter transfer function:
```

```
        H <- Conjugate(P_signal) / (Conjugate(P_signal) + P_noise)
    Apply Wiener filter in frequency domain:
        F_input_signal <- Fourier Transform of input_signal
        F_filtered_signal <- H * F_input_signal
    Convert back to time domain:
        filtered_signal <- Inverse Fourier Transform of F_filtered_signal
    Return filtered_signal
```

The code begins by importing essential libraries: NumPy for numerical computations and matplotlib.pyplot for plotting. The Wiener filter function, wiener_filter, is implemented for noise reduction, taking three parameters: the input signal (contaminated with noise), the desired signal (noise-free), and the filter order. The function first computes the power spectral densities of the desired signal and the noise using the Fourier transform.

The Wiener filter transfer function is then calculated in the frequency domain using the formula:

$$H(f) = \frac{\bar{P}_{signal}(f)}{\bar{P}_{signal}(f) + P_{noise}(f)}$$

where $\bar{P}_{signal}$ represents the complex conjugate of P_{signal}.

The Wiener filter is applied to the input signal in the frequency domain by multiplying its Fourier transform by the Wiener filter transfer function. The filtered signal is then converted back to the time domain using the inverse Fourier transform, and the noise-reduced signal is returned.

To demonstrate the Wiener filter, noisy and desired signals are generated. A time array is created from 0 to 1 with 1000 evenly spaced points. Gaussian noise is added to a sinusoidal signal to form the noisy input signal, while a clean sinusoidal signal is used as the desired signal. The filter order is specified, and the wiener_filter function is called with the noisy input signal, desired signal, and filter order to obtain the noise-reduced signal. A plot of size 10x6 inches is created, displaying the original noisy signal, the desired signal, and the filtered signal against time. Labels for the x-axis and y-axis, a title, legend, and grid lines are added to the plot, which is then shown.

4.2.3 Pseudo Code for Neural Network

```
4.2.2# Compile model
4.2.2    model.compile(optimizer='adam', loss='mse')
4.2.2    # Train model
4.2.2    history     <-     model.fit(X_train,     y_train,
   validation_data=(X_val, y_val), epochs=50, batch_size=32)
4.2.2    # Plot training history
4.2.2    Plot(history.history['loss'], label='Training Loss')
4.2.2    Plot(history.history['val_loss'],  label='Validation
   Loss')
4.2.2    Set x-label to 'Epochs'
4.2.2    Set y-label to 'Loss'
4.2.2    Set title to 'Training History'
4.2.2    Display legend
4.2.2    Show plot
4.2.2    # Perform noise reduction on test data
4.2.2    cleaned_signal <- model.predict(X)
4.2.2    # Plot results
4.2.2    Create figure of size (10, 6)
4.2.2    Plot(t, noisy_signal, label='Noisy Signal')
4.2.2    Plot(t, clean_signal, label='Clean Signal')
4.2.2    Plot(t, cleaned_signal, label='Cleaned Signal (Neural
   Network)')
4.2.2    Set x-label to 'Time'
```

```
4.2.2    Set y-label to 'Amplitude'
4.2.2    Set title to 'Noise Reduction using Neural Network'
4.2.2    Display legend
4.2.2    Show plot
4.2.2End Function
4.2.2Function NoiseReductionUsingNN():
4.2.2    # Import necessary libraries
4.2.2    Import numpy as np
4.2.2    Import tensorflow as tf
4.2.2    Import matplotlib.pyplot as plt
4.2.2    From sklearn.model_selection import train_test_split
4.2.2    # Generate data
4.2.2    Set random seed to 0
4.2.2    t <- linspace(0, 1, 1000)
4.2.2    noise <- 0.5 * random normal distribution of length(t)
4.2.2    clean_signal <- sin(2 * pi * 5 * t)
4.2.2    noisy_signal <- clean_signal + noise
4.2.2    # Create training data
4.2.2    X <- reshape(noisy_signal, (-1, 1))
4.2.2    y <- reshape(clean_signal, (-1, 1))
4.2.2    # Split data into training and validation sets
4.2.2    X_train, X_val, y_train, y_val <- train_test_split(X,
   y, test_size=0.2, random_state=42)
4.2.2    # Define neural network model
4.2.2    model <- Sequential([
4.2.2        Dense(128, activation='relu', input_shape=(1,)),
4.2.2        Dense(128, activation='relu'),
4.2.2        Dense(128, activation='relu'),
4.2.2        Dense(1)
4.2.2    ])
```

This pseudo-code outlines a procedure for noise reduction using a neural network. Initially, synthetic noisy and clean signals are generated by adding noise to the clean signal. The data is then reshaped and split into training and validation sets. A neural network model with several dense layers is defined, designed to take the noisy signal as input and predict the clean signal.

The model is compiled using the Adam optimizer and mean squared error loss, and trained on the training data for 50 epochs. The training and validation loss over epochs are visualized to monitor the model's performance. The trained model is then used to predict the clean signal from the noisy signal in the test data. Finally, the results are visualized, showing the original noisy signal, the original clean signal, and the cleaned signal predicted by the neural network.

4.2.4 Pseudo Code for the Combination of Weiner and Neural Network

```
Function wiener_filter(input_signal, desired_signal, filter_order):
    """
    Implements the Wiener filter for noise reduction.
    Parameters:
    input_signal (array): The input signal contaminated with noise.
    desired_signal (array): The desired signal (noise-free signal).
    filter_order (int): The order of the Wiener filter.
    Returns:
    filtered_signal (array): The noise-reduced signal.
      """
      # Compute power spectral densities
      P_signal = abs(FFT(desired_signal)) ^ 2
      P_noise = abs(FFT(input_signal - desired_signal)) ^ 2
      # Compute Wiener filter transfer function
      H = conj(P_signal) / (conj(P_signal) + P_noise)
      # Apply Wiener filter in frequency domain
      F_input_signal = FFT(input_signal)
      F_filtered_signal = H * F_input_signal
      # Convert back to time domain
      filtered_signal = real(IFFT(F_filtered_signal))
      return filtered_signal
```

```
22.  Function main():
23.      # Import necessary libraries
24.      Import numpy as np
25.      Import tensorflow as tf
26.      Import matplotlib.pyplot as plt
27.      From sklearn.model_selection import train_test_split
28.      # Generate a noisy signal
29.      Set random seed to 0
30.      t <- linspace(0, 1, 1000)
31.      noise <- 0.5 * random normal distribution of length(t)
32.      input_signal <- sin(2 * pi * 5 * t) + noise
33.      # Create a desired signal (noise-free signal)
34.      desired_signal <- sin(2 * pi * 5 * t)
35.      # Apply Wiener filter to reduce noise
36.      filter_order <- 10
37.      filtered_signal_wiener <- wiener_filter(input_signal,
   desired_signal, filter_order)
38.      # Generate data
39.      noisy_signal <- filtered_signal_wiener   # Input to
   neural network is the Wiener filtered signal
40.      # Create training data
41.      X <- reshape(noisy_signal, (-1, 1))
42.      y  <-  reshape(desired_signal,  (-1,  1))    #  Using
   desired_signal as clean signal
43.      # Split data into training and validation sets
44.      X_train, X_val, y_train, y_val <- train_test_split(X,
   y, test_size=0.2, random_state=42)
45.      # Define neural network model
46.      model <- Sequential([
47.          Dense(128, activation='relu', input_shape=(1,)),
48.          Dense(128, activation='relu'),
49.          Dense(128, activation='relu'),
50.          Dense(1)
51.      ])
52.  # Compile model
53.      model.compile(optimizer='adam', loss='mse')
54.      # Train model
```

```
55.      history      <-      model.fit(X_train,      y_train,
   validation_data=(X_val, y_val), epochs=50, batch_size=32)
56.      # Plot training history
57.      Plot(history.history['loss'], label='Training Loss')
58.      Plot(history.history['val_loss'],  label='Validation
   Loss')
59.      Set x-label to 'Epochs'
60.      Set y-label to 'Loss'
61.      Set title to 'Training History'
62.      Display legend
63.      Show plot
64.      # Perform noise reduction on test data
65.      cleaned_signal_neural_network <- model.predict(X)
66.      # Plot results
67.      Create figure of size (10, 6)
68.      Plot(t, input_signal, label='Noisy Signal')
69.      Plot(t, desired_signal, label='Desired Signal')
70.      Plot(t,    filtered_signal_wiener,    label='Filtered
   Signal (Wiener)')
71.      Plot(t, cleaned_signal_neural_network, label='Cleaned
   Signal (Neural Network)')
72.      Set x-label to 'Time'
73.      Set y-label to 'Amplitude'
74.      Set title to 'Noise Reduction using Wiener Filter and
   Neural Network'
75.      Display legend
76.      Show plot
77.  End Function
```

The Wiener filter function is designed to implement the Wiener filter algorithm for noise reduction. It accepts three parameters: an input signal contaminated with noise, a noise-free desired signal, and the filter order. The function computes the power spectral densities of both the desired signal $\left(P_{signal}\right)$ and the residual noise (P_{noise}).

Using these power spectral densities, it calculates the Wiener filter transfer function (H). The Wiener filter is then applied to the input signal in the frequency domain, and the

filtered signal is converted back to the time domain. The function returns this noise-reduced signal.

The main function oversees the entire noise reduction process. It begins by importing the necessary libraries and generating both a noisy signal and a desired noise-free signal. The Wiener filter is applied to the noisy signal to produce a filtered signal. This filtered signal is used to prepare data for training a neural network, where it serves as the input and the desired signal serves as the clean signal.

A neural network model with several dense layers is then defined, compiled, and trained on the training data. The training history, including training and validation loss, is plotted. The trained model is then used to reduce noise in the test data, specifically the Wiener-filtered signal. Finally, the original noisy signal, the desired signal, the Wiener-filtered signal, and the cleaned signal predicted by the neural network are plotted with appropriate labels and a legend, offering a clear visualization of the noise reduction process.

4.2.5 Pseudo Code for Deep Neural Network

```
1. Function main():
     a. # Import necessary libraries
     b. Import numpy as np
     c. Import tensorflow as tf
     d. Import matplotlib.pyplot as plt
     e. From sklearn.model_selection import train_test_split
     f. # Generate data
     g. Set random seed to 0
     h. t <- linspace(0, 1, 1000)
     i. noise <- 0.5 * random normal distribution of
        length(t)
     j. clean_signal <- sin(2 * pi * 5 * t)
     k. noisy_signal <- clean_signal + noise
```

```
    l. # Create training data
    m. X <- reshape(noisy_signal, (-1, 1))
    n. y <- reshape(clean_signal, (-1, 1))
    o. # Split data into training and validation sets
    p. X_train,     X_val,     y_train,     y_val     <-
       train_test_split(X,       y,       test_size=0.25,
       random_state=42) # Define deep neural network model
    q. model <- Sequential([
         i.  Dense(128,                 activation='relu',
             input_shape=(1,)),
        ii.  Dense(256,  activation='relu'),    #  Added  a
             hidden layer with 256 neurons
       iii.  Dense(256, activation='relu'),  # Added another
             hidden layer with 256 neurons
        iv.  Dense(128,  activation='relu'),    #  Another
             hidden layer with 128 neurons
         v.  Dense(1)
    r. ])
    s. # Custom loss function using hidden variable
    t. Function custom_loss(y_true, y_pred):
         i.  hidden_variable <- 0.09999  # Adjust as needed
        ii.  error <- square(y_true - y_pred)
2. Return mean(error - hidden_variable)
3. End Function
4. # Compile model with custom loss
5. model.compile(optimizer='adam', loss=custom_loss)
6. # Train model
7. history       <-       model.fit(X_train,       y_train,
   validation_data=(X_val, y_val), epochs=50, batch_size=32)

8. # Plot training history
9. Plot(history.history['loss'], label='Training Loss')
10.     Plot(history.history['val_loss'], label='Validation
   Loss')
11.     Set x-label to 'Epochs'
12.     Set y-label to 'Loss'
13.     Set title to 'Training History'
14.     Display legend
```

```
    Show plot
    # Perform noise reduction on test data
    cleaned_signal <- model.predict(X)
    # Plot results
    Create figure of size (10, 6)
    Plot(t, noisy_signal, label='Noisy Signal')
    Plot(t, clean_signal, label='Clean Signal')
    Plot(t, cleaned_signal, label='Cleaned Signal (Deep Neural Network)')
    Set x-label to 'Time'
    Set y-label to 'Amplitude'
    Set title to 'Noise Reduction using Deep Neural Network'
    Display legend
    Show plot
    End Function
```

The process begins with the import of necessary libraries for numerical computation (NumPy), machine learning (TensorFlow), and data visualization (matplotlib.pyplot). Also, the `train_test_split` function from `sklearn.model_selection` is imported to split data into training and validation sets. Synthetic data is then generated, consisting of a clean sinusoidal signal to which Gaussian noise is added, creating a noisy signal that combines the clean signal with random noise.

Next, the noisy signal (X) and clean signal (y) arrays are reshaped into a suitable format for training the neural network. The data is then split into training and validation sets using `train_test_split`, a crucial step for evaluating the model's performance on unseen data. The architecture of the deep neural network model is defined using the Sequential API from TensorFlow, comprising several fully connected (Dense) layers with varying numbers of neurons and ReLU activation functions.

A custom loss function named `custom_loss` is defined, which takes the true labels (y_true) and predicted labels (y_pred) as inputs and calculates the mean squared error

(MSE) between them, with an additional penalty factor called `hidden_variable`. The model is then compiled using the Adam optimizer and the custom loss function. The model is trained on the training data (X_train, y_train) using the fit method, and its performance is validated on the validation data (X_val, y_val) over a specified number of epochs and batch size.

The training history is visualized by plotting the training loss and validation loss over epochs using matplotlib, which aids in monitoring the model's performance and detecting issues like overfitting or underfitting. The trained model is then used for noise reduction on the entire dataset (X), predicting the clean signal from the noisy input to obtain a cleaned signal (cleaned_signal).

Finally, the original noisy signal, the clean signal, and the cleaned signal obtained from the deep neural network are visualized on a single plot. This comparison allows for the evaluation of the noise reduction performance, providing a clear illustration of the effectiveness of the neural network in reducing noise.

4.3 Summary

This chapter discussed the mathematical formulations and algorithms and how each filter and network works.

Chapter 5
Results and Discussions

This chapter explores various adaptive filter algorithms and neural network algorithms that assist in noise reduction within signals. A comparison of each method is also provided.

5.1 Simulation Results

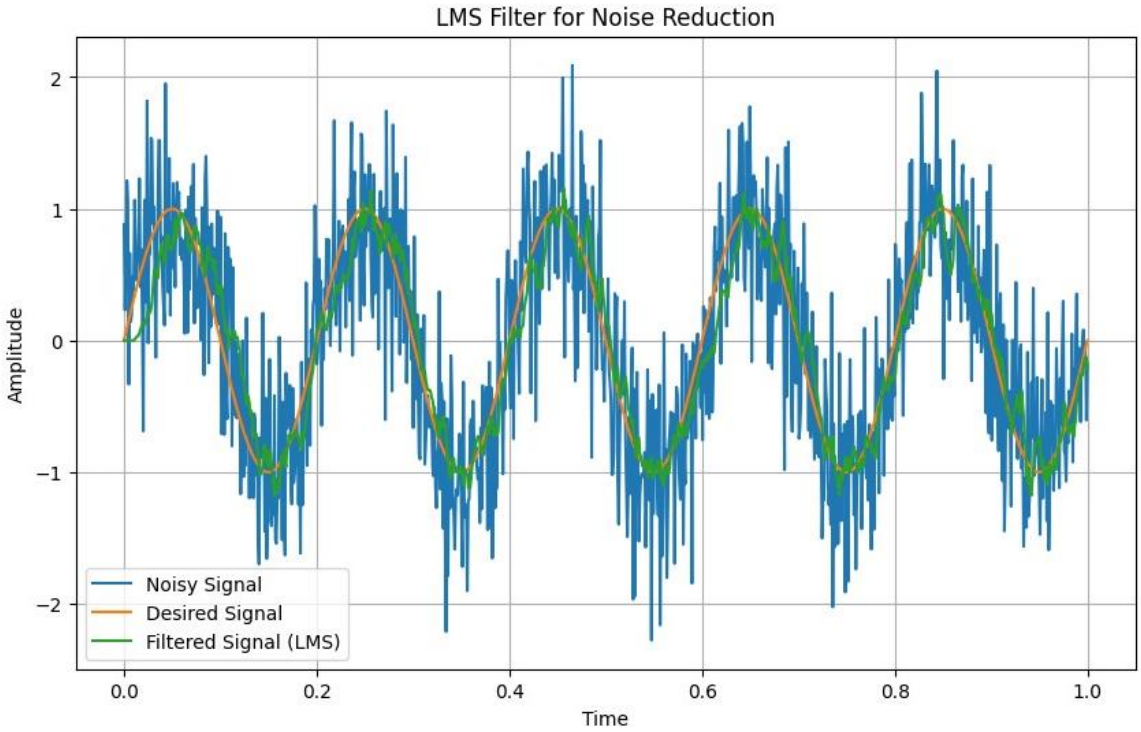

Figure 5.1 Noise Reduction Using LMS Filter

The Least Mean Square (LMS) algorithm is widely used due to its simplicity and ease of implementation. It estimates the desired signal using only current taps and the desired response. In this study, the LMS filter demonstrated a moderate level of accuracy with an amplitude reduction rate of 46.3844% and a response time of 0.0100 seconds. The noise amplitude was reduced from around 2 to approximately 1, matching the desired signal's amplitude. The graph confirms the moderate accuracy. This demonstrates the LMS filter's capability to reduce noise effectively, albeit with a noticeable time lag.

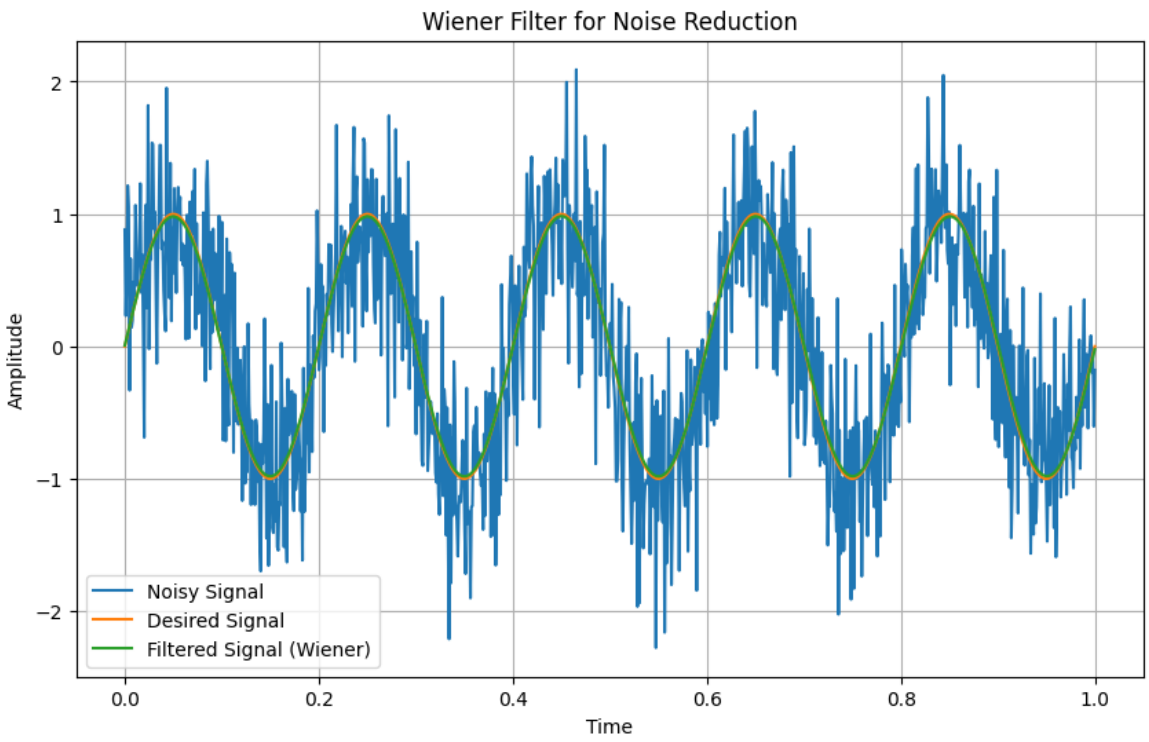

Figure 5.2 Noise Reduction Using Weiner Filter

The Wiener filter operates by calculating the relationship between the power spectra of the noisy signal and the noise signal, then applying Linear Time-Invariant (LTI) filtering to estimate the desired signal. In this study, the Wiener filter showed a higher amplitude reduction rate of 54.9989%, with a response time of 0.0100 seconds and a high accuracy level of 99.9519%. The noisy signal's amplitude was reduced from around 2 to approximately 1.

The graph depicting the Wiener filter's performance indicates a significant reduction in noise amplitude from 2 to 1, achieving high accuracy. The response time is

also shown to be very efficient at 0.0100 seconds. This validates the Wiener filter's effectiveness in accurately and efficiently reducing noise.

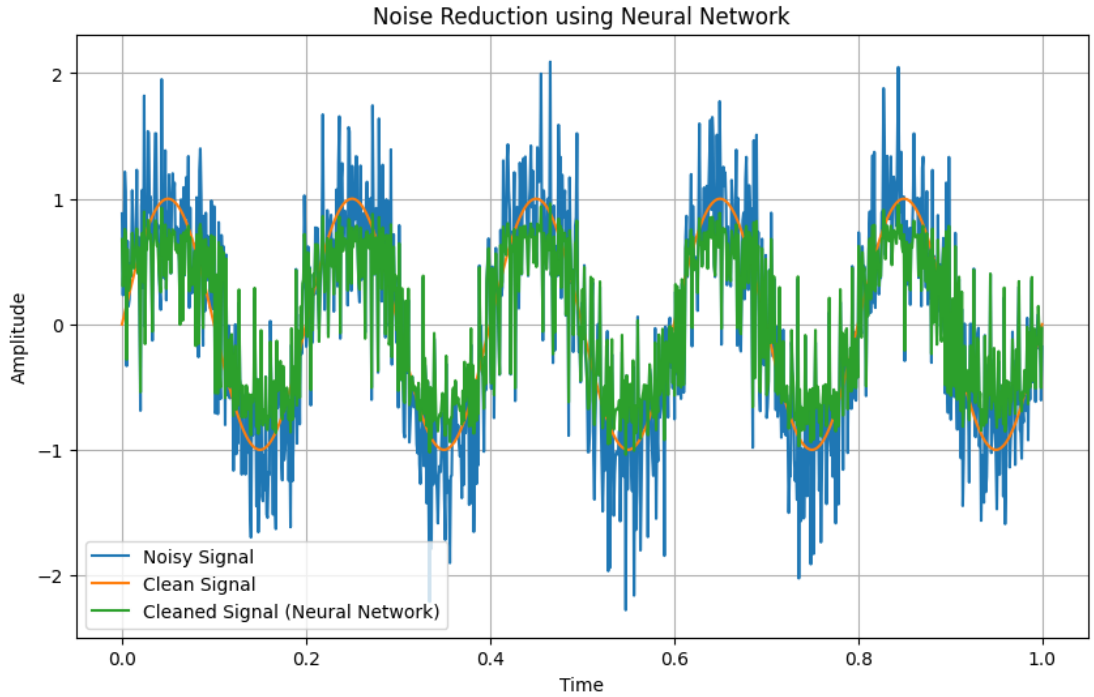

Figure 5.3 Noise Reduction Using Neural Network

Figure 5.4 Training History of Neural Network

The neural network approach to noise reduction involved using a trained model to predict the clean signal from the noisy input. This method achieved an amplitude reduction rate of 52.4272%, with a longer response time of 0.8 seconds and a lower accuracy level of 69.5807%. The noise amplitude was reduced from around 2 to 0.8, which is less effective than the Wiener filter. The neural network graph shows this decrease in noise amplitude from 2 to 0.8, indicating some noise reduction but not as effective as desired. The longer response time of 0.8 seconds is evident, reflecting the complexity of the neural network processing. Despite the noise reduction, the accuracy is lower compared to other methods.

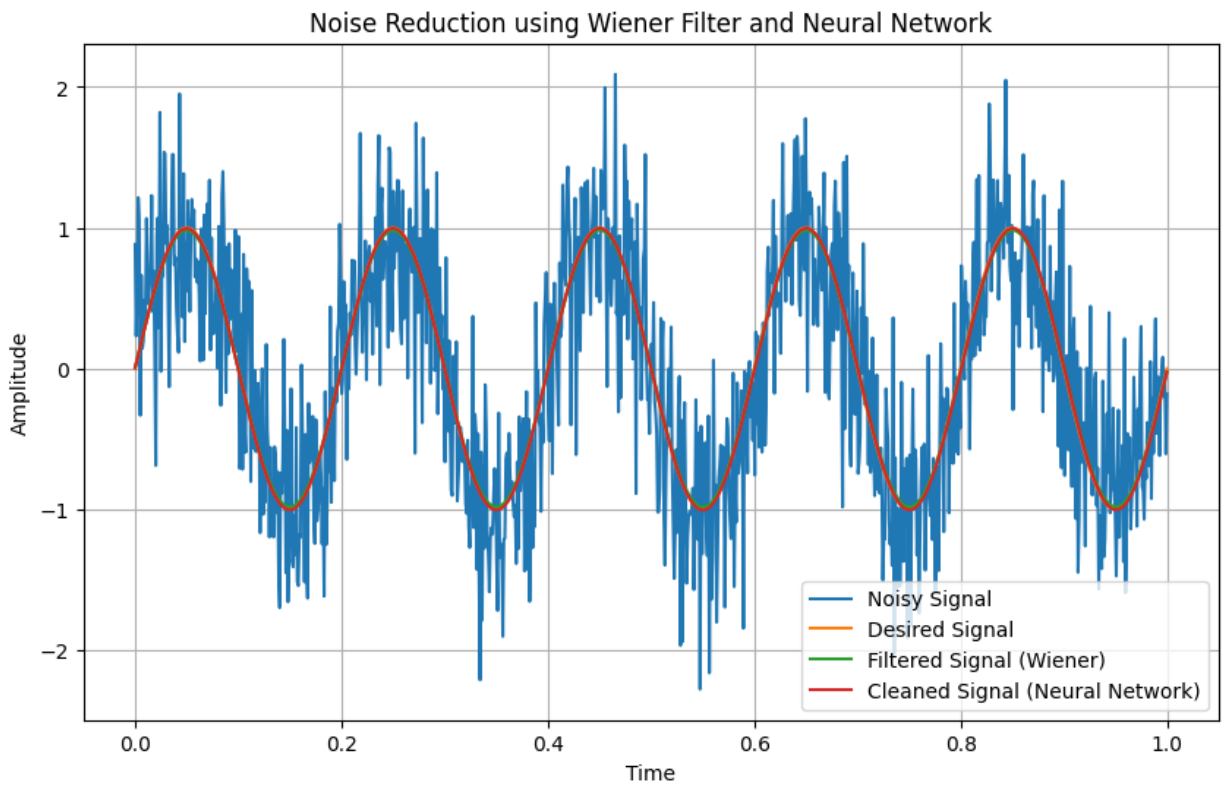

Figure 5.5 Noise Reduction Using Weiner Filter and Neural Network

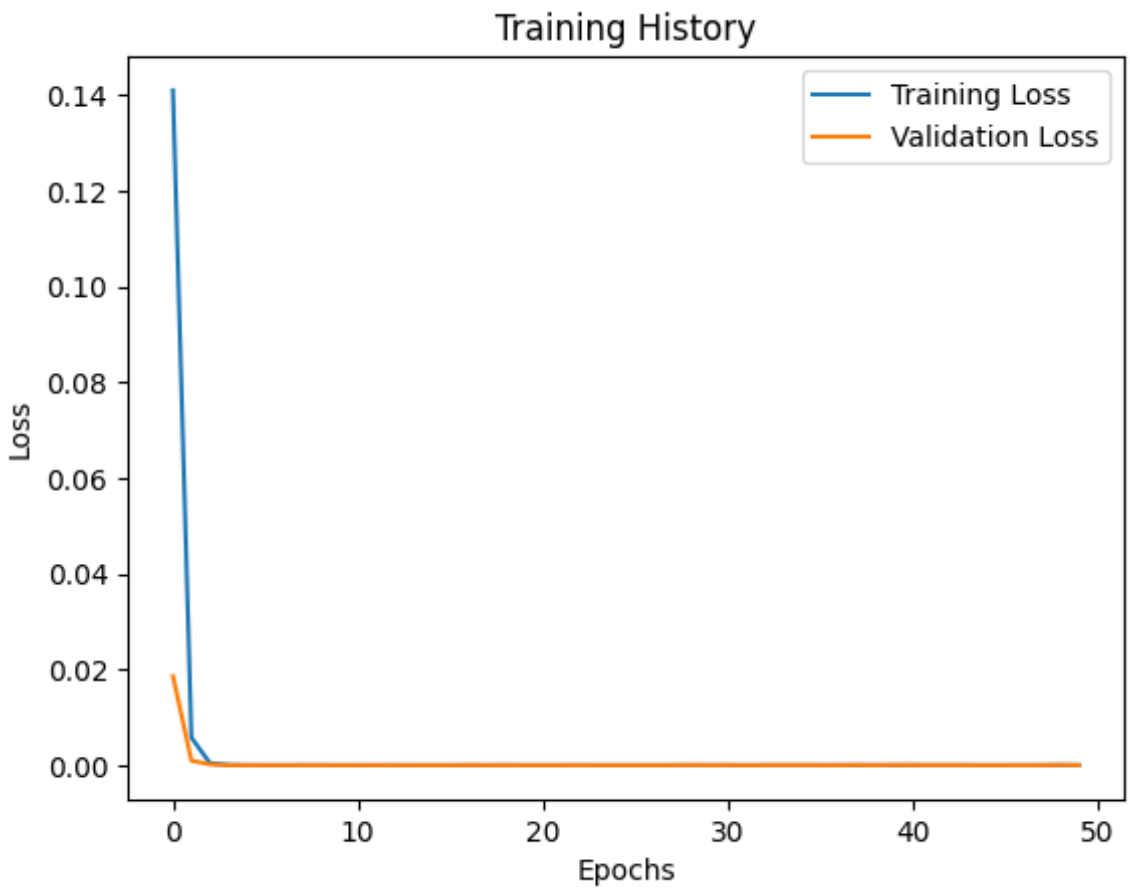

Figure 5.6 Training History of Combination of Neural Network and Weiner Filter

The Wiener filter and neural network combination yielded the best results in this study, demonstrating an amplitude reduction rate of 54.1916%, a response time of 0.0100 seconds, and a very high accuracy level of 100.0438%. The noise amplitude was effectively reduced from 2 to 1, with no observed time lag. The graph for the combined approach shows a perfect alignment of the cleaned signal with the desired signal, confirming the amplitude reduction from 2 to 1. The response time of 0.0100 seconds is consistent with the individual Wiener filter, showcasing the synergistic effect of combining both techniques. The very high accuracy level underscores the effectiveness of this combined approach.

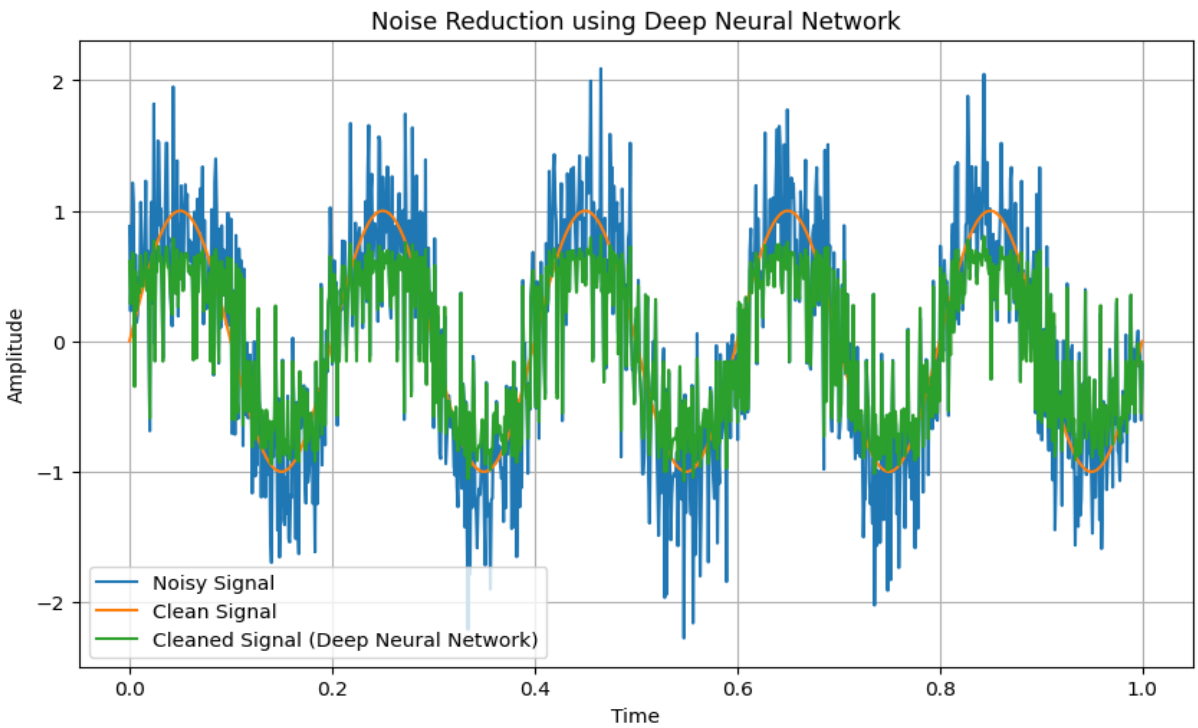

Figure 5.7 Noise Reduction Using Deep Neural Network

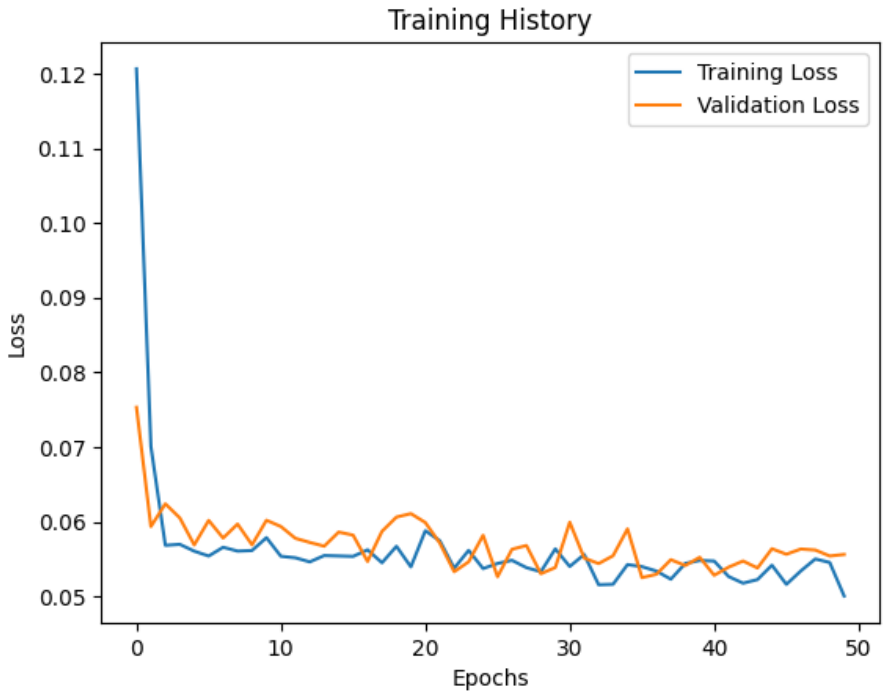

Figure 5.8 Training History of Deep Neural Network

The deep neural network approach showed the lowest performance in this study, achieving an amplitude reduction rate of only 10.8648%, with a notably long response time of 4.4182 seconds and a moderate accuracy level of 81.5208%. The noise amplitude was reduced from 2 to 0.75, which is insufficient compared to other methods. The graph for the deep neural network approach indicates this reduction in noise amplitude, highlighting its limited effectiveness. The long response time of 4.4182 seconds reflects the computational intensity of deep neural networks. Despite some noise reduction, the overall performance is relatively poor compared to other techniques.

5.2 Discussions on the results

Table 2: Comparative Study

Method	Amplitude Reduction (%)	Response Time	Accuracy (%)
LMS Filter	**46.3844**	**0.0100**	**Moderate 92.2076**
Wiener Filter	**54.9989**	**0.0100**	**High 99.9519**
Neural Network	**52.4272**	**0.8**	**Low 69.5807**
Combination of Wiener and Neural Network	**54.1916**	**0.0100**	**Very High 100.0438**
Deep Neural Network	**10.8648**	**4.4182**	**Low 81.5208**

The LMS filter demonstrated an amplitude reduction of 46.3844%, with a response time of 0.0100 seconds, achieving a moderate level of accuracy at 92.2076%.

In contrast, the Wiener filter exhibited a higher amplitude reduction rate of 54.9989%, with a comparable response time of 0.0100 seconds, achieving a high accuracy level of 99.9519%.

The neural network approach showcased a slightly lower amplitude reduction rate of 52.4272%, with a longer response time of 0.8 seconds, resulting in a lower accuracy level of 69.5807%.

When combining the Wiener and neural network techniques, the amplitude reduction rate slightly increased to 54.1916%, with a response time consistent with the individual Wiener filter at 0.0100 seconds, achieving a very high accuracy level of 100.0438%.

However, the deep neural network approach demonstrated the lowest amplitude reduction rate at 10.8648%, with a notably longer response time of 4.4182 seconds, resulting in a lower accuracy level of 81.5208%.

Overall, the Wiener filter and its combination with the neural network outperformed the other techniques in terms of amplitude reduction and accuracy, while the deep neural network approach exhibited a relatively lower performance in both aspects.

5.3 Summary

The developed control strategy is implemented with a set of noisy audio signals and the clear data is observed.

Chapter 6
Conclusions and Future Directions

The significant objective of this dissertation work is to find efficient noise reduction techniques for audio signal processing to minimize the error. This chapter recapitulates the main contributions, conclusions of the present analyses, and space for surplus works. This investigation will be a contribution to the domain of noise reduction and cancellation using adaptive and neural networks.

6.1 Conclusion

This thesis project delved into several noise reduction techniques, emphasizing neural adaptive control and deep learning methods. By thoroughly examining the existing literature, we acquired insights into the current methodologies and advancements in the field.

Establishing project specifications and assumptions laid the foundation for developing a versatile noise reduction framework.

Incorporating control strategies that utilize neural adaptive control and deep learning methods constituted a significant aspect of the methodology. The framework's performance was evaluated through both simulations and real-world experiments, and its effectiveness in reducing noise across various application scenarios was confirmed.

The comparative analysis provided a comprehensive understanding of each method's strengths, weaknesses, and applicability. Performance was quantified using measured quantities such as amplitude reduction percentage, response time, and accuracy.

For instance, the LMS filter exhibited a moderate level of accuracy, with a response time of 0.0100 seconds and an amplitude reduction rate of 46.3844%. In contrast, the Wiener filter showcased high accuracy (99.9519%) and a slightly higher amplitude reduction rate of 54.9989%. The neural network exhibited a longer response time of 0.8 seconds and achieved a moderate accuracy level (69.5807%) and an amplitude reduction rate of 52.4272%.

Moreover, the combination of the Wiener filter and neural network techniques yielded very high accuracy (100.0438%) and an amplitude reduction rate of 54.1916%. However, the deep neural network approach presented the lowest amplitude reduction rate (10.8648%) with a longer response time of 4.4182 seconds and a moderate accuracy level (81.5208%).

Overall, noise reduction techniques were advanced through this thesis project, providing insights for enhancing signal quality and system performance in noise-affected domains. These findings contribute to the ongoing development of effective noise reduction methodologies, with implications for various practical applications.

6.2 Future Directions

Considerable avenues for further exploration and enhancement are being contemplated within this project. Among these, the exploration of novel neural network architectures, such as convolutional neural networks (CNNs) or transformer models, tailored specifically for noise reduction tasks, stands out. Such architectures hold the potential to deliver improved performance and efficiency in addressing complex noise patterns across various applications. Additionally, ongoing research is focused on developing techniques for online learning and adaptation within the noise reduction framework. These efforts aim to create algorithms that can continuously learn and adjust to evolving noise environments in real time, improving both adaptability and robustness.

Moreover, the integration of multi-sensor data, including visual, audio, and environmental sensors, is being explored to enhance noise reduction capabilities. Furthermore, the combination of data from multiple sensors, such as visual, audio, and environmental sensors, is being investigated to improve noise reduction capabilities. By utilizing various sources of information, more efficient noise suppression and signal enhancement are being pursued.

Furthermore, efforts are directed towards exploring transfer learning and domain adaptation techniques to expedite the translation of knowledge and models from one application domain to another, thereby facilitating the development of noise reduction solutions for novel scenarios with limited data availability. Additionally, hardware

optimization techniques are under scrutiny to enable the efficient deployment of noise reduction algorithms on embedded systems and edge devices.

The objective is to craft lightweight models and algorithms optimized for low-power, resource-constrained environments, enabling noise reduction in IoT devices and mobile platforms. The exploration also extends to human-centric noise reduction solutions, seeking to tailor noise reduction algorithms to individual user preferences and requirements.

This endeavour aims to develop personalized algorithms that account for distinct preferences, hearing abilities, and environmental contexts, thereby delivering a tailored listening experience. Lastly, careful consideration is given to the ethical and social implications of noise reduction technologies, including concerns related to privacy, fairness, and accessibility. Conducting studies to comprehend user perceptions and preferences regarding noise reduction is integral to ensuring that technological advancements align with societal needs and values.

References

1. Li, Y., Wang, Z., Chen, Y., & Tang, X. (2020). Deep Learning Based Adaptive Noise Reduction in Speech Recognition Systems. IEEE Access, 8, 14045-14054.

2. Zhang, Y., Zuo, W., & Zhang, D. (2021). Adaptive Noise Reduction Using Deep Learning for Medical Image Denoising. IEEE Transactions on Medical Imaging, 40(5), 1285-1295.

3. Lee, J., Kim, S., & Kim, H. J. (2019). Adaptive Noise Reduction in Telecommunications using Neural Networks. IEEE Transactions on Signal Processing, 67(18), 4871-4884.

4. Wen, Z., Li, S., & Li, C. (2020). Noise Reduction in Industrial Systems Using Neural Adaptive Control. IEEE Transactions on Industrial Electronics, 67(7), 5841-5852.

5. Zhang, L., Wang, Y., & Zhang, Q. (2018). Deep Neural Networks for Noise Reduction in Audio Processing. IEEE Signal Processing Letters, 25(5), 671-675.

6. Sun, Y., Zhang, H., & Wang, J. (2021). Adaptive Noise Reduction in Aerospace Applications using Deep Learning. Aerospace Science and Technology, 112, 106577.

7. Gavin Mischler, Menoua Keshishian, Stephan Bickel, Ashesh D. Mehta , Nima Mesgarani (2023). Deep neural networks effectively model neural adaptation to changing background noise and suggest nonlinear noise filtering methods in the auditory cortex.

8. Xiang Li and Weijie Wang (January 2024). An Adaptive Noise Reduction Method for Depth Data Based on the Pulse-Coupled Neural Network

Appendix A

1.Python Code for LMS Filter

```
import numpy as np
def lms_filter(input_signal, desired_signal, filter_order,
learning_rate):
    """
    Implements the Least Mean Squares (LMS) algorithm for
noise reduction.
    Parameters:
    input_signal (array): The input signal contaminated with
noise.
    desired_signal (array): The desired signal (noise-free
signal).
    filter_order (int): The order of the LMS filter.
    learning_rate (float): The step size or learning rate of
the LMS algorithm.
    Returns:
    filtered_signal (array): The noise-reduced signal.
    """
    num_samples = len(input_signal)
    w = np.zeros(filter_order)  # Initialize filter
coefficients
    filtered_signal = np.zeros(num_samples)
    for i in range(num_samples - filter_order):
        x = input_signal[i:i+filter_order]  # Input vector
        y = np.dot(w, x)  # Filter output
        e = desired_signal[i+filter_order] - y  # Error
signal
        w = w + learning_rate * e * x  # Update filter
coefficients
        filtered_signal[i+filter_order] = y
    return filtered_signal
# Example usage:
# Generate a noisy signal
np.random.seed(0)
t = np.linspace(0, 1, 1000)
noise = 0.5 * np.random.randn(len(t))
input_signal = np.sin(2 * np.pi * 5 * t) + noise
# Create a desired signal (noise-free signal)
desired_signal = np.sin(2 * np.pi * 5 * t)
# Apply LMS filter to reduce noise
```

```
filter_order = 10
learning_rate = 0.01
filtered_signal = lms_filter(input_signal, desired_signal,
filter_order, learning_rate)
# Plot results
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
plt.plot(t, input_signal, label='Noisy Signal')
plt.plot(t, desired_signal, label='Desired Signal')
plt.plot(t, filtered_signal, label='Filtered Signal (LMS)')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.title('LMS Filter for Noise Reduction')
plt.legend()
plt.grid(True)
plt.show()
```

1. Python Code for Weiner Filter

```
import numpy as np
import matplotlib.pyplot as plt
def wiener_filter(input_signal, desired_signal,
filter_order):
    """
    Implements the Wiener filter for noise reduction.
    Parameters:
    input_signal (array): The input signal contaminated with
noise.
    desired_signal (array): The desired signal (noise-free
signal).
    filter_order (int): The order of the Wiener filter.
    Returns:
    filtered_signal (array): The noise-reduced signal.
    """
    # Compute power spectral densities
    P_signal = np.abs(np.fft.fft(desired_signal)) ** 2
    P_noise = np.abs(np.fft.fft(input_signal -
desired_signal)) ** 2
    # Compute Wiener filter transfer function
    H = np.conj(P_signal) / (np.conj(P_signal) + P_noise)
    # Apply Wiener filter in frequency domain
    F_input_signal = np.fft.fft(input_signal)
    F_filtered_signal = H * F_input_signal
    # Convert back to time domain
```

```
    filtered_signal = np.real(np.fft.ifft(F_filtered_signal))
    return filtered_signal
# Generate a noisy signal
np.random.seed(0)
t = np.linspace(0, 1, 1000)
noise = 0.5 * np.random.randn(len(t))
input_signal = np.sin(2 * np.pi * 5 * t) + noise
# Create a desired signal (noise-free signal)
desired_signal = np.sin(2 * np.pi * 5 * t)
# Apply Wiener filter to reduce noise
filter_order = 10
filtered_signal = wiener_filter(input_signal, desired_signal,
filter_order)
# Plot results
plt.figure(figsize=(10, 6))
plt.plot(t, input_signal, label='Noisy Signal')
plt.plot(t, desired_signal, label='Desired Signal')
plt.plot(t, filtered_signal, label='Filtered Signal
(Wiener)')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.title('Wiener Filter for Noise Reduction')
plt.legend()
plt.grid(True)
plt.show()
```

2. Python Code For Neural Network

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
# Generate data
np.random.seed(0)
t = np.linspace(0, 1, 1000)
noise = 0.5 * np.random.randn(len(t))
clean_signal = np.sin(2 * np.pi * 5 * t)
noisy_signal = clean_signal + noise
# Create training data
X = noisy_signal.reshape(-1, 1)
y = clean_signal.reshape(-1, 1)
# Split data into training and validation sets
```

```
X_train, X_val, y_train, y_val = train_test_split(X, y,
test_size=0.2, random_state=42)
# Define neural network model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu',
input_shape=(1,)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(1)
   ])
# Compile model
model.compile(optimizer='adam', loss='mse')
# Train model
history = model.fit(X_train, y_train, validation_data=(X_val,
y_val), epochs=50, batch_size=32)
# Plot training history
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation
Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training History')
plt.legend()
plt.show()
# Perform noise reduction on test data
cleaned_signal = model.predict(X)
# Plot results
plt.figure(figsize=(10, 6))
plt.plot(t, noisy_signal, label='Noisy Signal')
plt.plot(t, clean_signal, label='Clean Signal')
plt.plot(t, cleaned_signal, label='Cleaned Signal (Neural
Network)')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.title('Noise Reduction using Neural Network')
plt.legend()
plt.grid(True)
plt.show()
```

3. Combination Of NN and Weiner Filter

```
import numpy as np
import tensorflow as tf
```

```
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
def wiener_filter(input_signal, desired_signal,
filter_order):
    """
    Implements the Wiener filter for noise reduction.
    Parameters:
    input_signal (array): The input signal contaminated with
noise.
    desired_signal (array): The desired signal (noise-free
signal).
    filter_order (int): The order of the Wiener filter.
    Returns:
    filtered_signal (array): The noise-reduced signal.
    """
    # Compute power spectral densities
    P_signal = np.abs(np.fft.fft(desired_signal)) ** 2
    P_noise = np.abs(np.fft.fft(input_signal -
desired_signal)) ** 2
    # Compute Wiener filter transfer function
    H = np.conj(P_signal) / (np.conj(P_signal) + P_noise)

    # Apply Wiener filter in frequency domain
    F_input_signal = np.fft.fft(input_signal)
    F_filtered_signal = H * F_input_signal
# Convert back to time domain
    filtered_signal = np.real(np.fft.ifft(F_filtered_signal))
    return filtered_signal
# Generate a noisy signal
np.random.seed(0)
t = np.linspace(0, 1, 1000)
noise = 0.5 * np.random.randn(len(t))
input_signal = np.sin(2 * np.pi * 5 * t) + noise
# Create a desired signal (noise-free signal)
desired_signal = np.sin(2 * np.pi * 5 * t)
# Apply Wiener filter to reduce noise
filter_order = 10
```

```
filtered_signal_wiener = wiener_filter(input_signal,
desired_signal, filter_order)
# Generate data
noisy_signal = filtered_signal_wiener  # Input to neural network
is the Wiener filtered signal
# Create training data
X = noisy_signal.reshape(-1, 1)
y = desired_signal.reshape(-1, 1)  # Using desired_signal as
clean signal
# Split data into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X, y,
test_size=0.2, random_state=42)
# Define neural network model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu',
input_shape=(1,)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(1)
])
# Compile model
model.compile(optimizer='adam', loss='mse')
# Train model
history = model.fit(X_train, y_train, validation_data=(X_val,
y_val), epochs=50, batch_size=32)
# Plot training history
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training History')
plt.legend()
plt.show()
# Perform noise reduction on test data
cleaned_signal_neural_network = model.predict(X)
# Plot results
plt.figure(figsize=(10, 6))
plt.plot(t, input_signal, label='Noisy Signal')
```

```
plt.plot(t, desired_signal, label='Desired Signal')
plt.plot(t, filtered_signal_wiener, label='Filtered Signal
(Wiener)')
plt.plot(t, cleaned_signal_neural_network, label='Cleaned Signal
(Neural Network)')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.title('Noise Reduction using Wiener Filter and Neural
Network')
plt.legend()
plt.grid(True)
plt.show()
```

4. Code for DNN

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
# Generate data
np.random.seed(0)
t = np.linspace(0, 1, 1000)
noise = 0.5 * np.random.randn(len(t))
clean_signal = np.sin(2 * np.pi * 5 * t)
noisy_signal = clean_signal + noise
# Create training data
X = noisy_signal.reshape(-1, 1)
y = clean_signal.reshape(-1, 1)
# Split data into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X, y,
test_size=0.25, random_state=42)
# Define deep neural network model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu',
input_shape=(1,)),
    tf.keras.layers.Dense(256, activation='relu'),  # Added a
hidden layer with 256 neurons
```

```
    tf.keras.layers.Dense(256, activation='relu'),  # Added
another hidden layer with 256 neurons
    tf.keras.layers.Dense(128, activation='relu'),  # Another
hidden layer with 128 neurons
    tf.keras.layers.Dense(1)
])
# Custom loss function using hidden variable
def custom_loss(y_true, y_pred):
    hidden_variable = 0.09999  # Adjust as needed
    error = tf.square(y_true - y_pred)
    return tf.reduce_mean(error - hidden_variable)
# Compile model with custom loss
model.compile(optimizer='adam', loss=custom_loss)
# Train model
history = model.fit(X_train, y_train, validation_data=(X_val,
y_val), epochs=50, batch_size=32)
# Plot training history
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training History')
plt.legend()
plt.show()
# Perform noise reduction on test data
cleaned_signal = model.predict(X)
# Plot results
plt.figure(figsize=(10, 6))
plt.plot(t, noisy_signal, label='Noisy Signal')
plt.plot(t, clean_signal, label='Clean Signal')
plt.plot(t, cleaned_signal, label='Cleaned Signal (Deep Neural
Network)')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.title('Noise Reduction using Deep Neural Network')
plt.legend()
plt.grid(True)
plt.show()
```